

Pebble Weighted Automata and Weighted Logics

BENEDIKT BOLLIG, PAUL GASTIN, BENJAMIN MONMEGE, LSV, ENS Cachan, CNRS, Inria
MARC ZEITOUN, Université de Bordeaux, LaBRI, CNRS, Inria

We introduce new classes of weighted automata on words. Equipped with pebbles, they go beyond the class of recognizable formal power series: they capture weighted first-order logic enriched with a quantitative version of transitive closure. In contrast to previous work, this calculus allows for unrestricted use of existential and universal quantifications over positions of the input word. We actually consider both two-way and one-way pebble weighted automata. The latter class constrains the head of the automaton to walk left-to-right, resetting it each time a pebble is dropped. Such automata have already been considered in the boolean setting, in the context of data words. Our main result states that two-way pebble weighted automata, one-way pebble weighted automata, and our weighted logic are expressively equivalent. We also give new logical characterizations of standard recognizable series.

Categories and Subject Descriptors: F.3.1 [Logics and Meanings of Programs]: Specifying and Verifying and Reasoning about Programs; F.4.3 [Mathematical Logic and Formal Languages]: Formal Languages

General Terms: Algorithms, Languages, Theory, Verification

Additional Key Words and Phrases: Formal power series, Pebble automata, Quantitative properties, Weighted automata, Weighted logics

1. INTRODUCTION

Connections between logical and state-based formalisms have always been a fascinating research area in theoretical computer science, which produced some fundamental theorems. The line of classical results started with the equivalence of MSO logic and finite automata [Büchi 1960; Elgot 1961; Trakhtenbrot 1961] and gave rise to natural generalizations to trees in terms of logics for tree automata [Thatcher and Wright 1968; Rabin 1969; Comon et al. 2008], tree-walking automata, and pebble tree automata [Bojańczyk et al. 2006; Samuelides and Segoufin 2007; Bojańczyk 2008]. Such automata models recently attracted significant interest, for instance in the context of manipulating XML documents and evaluating XPath queries [ten Cate and Segoufin 2010].

Other extensions of finite automata are of quantitative nature and include timed automata, probabilistic systems, and transducers, which all come with more or less natural, specialized logical characterizations. A generic concept of adding weights to qualitative systems is provided by the theory of weighted automata [Droste et al. 2009], first introduced by Schützenberger [Schützenberger 1961]. The output of a weighted automaton running on

This is an extended and enhanced version of results published in [Bollig et al. 2010]. This paper was written when the last author was visiting LSV as a full time Inria researcher.

This work was partly supported by ANR 2010 BLAN 0202 01 FREC, ARCUS Île-de-France-Inde, and LIA InForMel.

Authors' addresses: B. Bollig, P. Gastin, and B. Monmege: LSV, ENS Cachan, CNRS & Inria, 61, Av. du Président Wilson, 94235 Cachan Cedex, France. Email: {bollig,gastin,monmege}@lsv.ens-cachan.fr; M. Zeitoun: LaBRI, Université de Bordeaux, CNRS & Inria, 351 cours de la Libération, 33405 Talence Cedex, France. Email: mz@labri.fr.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© YYYY ACM 1529-3785/YYYY/01-ARTA \$15.00

DOI: <http://dx.doi.org/10.1145/0000000.0000000>

a word is no longer a boolean value discriminating between accepted and rejected behaviors. A word is rather mapped to a weight from a semiring, summing over the weights of all possible runs, each calculated as the product of its transition outcomes. Indeed, probabilistic automata and word transducers appear as instances of that framework, which found its way into numerous application areas such as natural language processing and speech recognition or digital image compression (see [Droste et al. 2009, Part IV]).

A logical characterization of weighted automata, however, was established only recently [Droste and Gastin 2009], in terms of a (restricted) weighted MSO logic capturing the recognizable formal power series (i.e., the behaviors of finite weighted automata). The key idea is to interpret existential and universal quantifications as the operations sum and product from a semiring. The original work on a logical characterization of the recognizable series on words has been extended to more general structures such as ranked and unranked trees [Droste and Vogler 2006; Droste and Vogler 2011], nested words [Mathissen 2010], pictures [Fichtner 2011], and Mazurkiewicz traces [Meinecke 2006], to mention only a few. In all these cases, however, one has to restrict the universal first-order quantification to stay within the class of recognizable series.

In the present paper, we follow a different approach. Instead of restricting the logic, we define an extended automaton model that naturally *reflects* it. We will indeed show that universal quantification as interpreted in [Droste and Gastin 2009] is essentially captured by a pebble (two-way) mechanism in the automata-theoretic counterpart. Inspired by the theory of two-way and pebble automata on words and trees [Rabin and Scott 1959; Engelfriet and Hoogetboom 1999; Bojańczyk et al. 2006; Samuelides and Segoufin 2007; Bojańczyk 2008], we actually define weighted generalizations that preserve their natural connections with logic. More precisely, we introduce pebble weighted automata on words and establish expressive equivalence to weighted first-order logic with a quantitative extension of the boolean transitive closure, extending the classical boolean case for words [Engelfriet and Hoogetboom 2007]. Our equivalence proof makes a detour via one-way pebble weighted automata, which were already considered in the boolean context for infinite alphabets [Neven et al. 2004] and restrict pebble weighted automata to left-to-right runs. Note that fragments of our logic also yield alternative characterizations of the (classical) recognizable formal power series.

Related work. There has been a wide range of quantitative extensions of first-order and temporal logic that are quite different from the logics studied in this paper. Most of them are specialized and cannot be instantiated by arbitrary semirings.

The formalisms from [Immerman and Lander 1990; Etessami 1997; Libkin 2000; Hella et al. 2001] consider counting and aggregate quantifiers for first-order logic to talk about two-sorted structures. Roughly speaking, they allow us to specify how often a boolean formula shall be satisfied, or to sum up over all elements satisfying a property. However, a sentence still defines a language (of two-sorted structures) rather than a formal power series. Moreover, the focus in these works has been on (non-)definability and locality properties in the sense of Hanf and Gaifman. Connections with automata (not to mention weighted automata) were not studied, and there seems to be no immediate correspondence.

When weighted monadic second-order logic is provided with a threshold operator, it embeds probabilistic CTL [Hansson and Jonsson 1994], as observed in [Bollig and Gastin 2009]. In principle, this also applies to counting versions of the temporal logics CTL and LTL [Laroussinie et al. 2010a; Laroussinie et al. 2010b], which can reason about how often a formula is satisfied along a path. Again, a formula from these logics defines a language of words or trees rather than a series, which leads to classical (boolean) satisfiability and model checking questions. Contrary to this, we are aiming in this paper at a Büchi-like correspondence of logic and automata for *formal power series*, which associate a *quantity* with every word.

A *quantitative* semantics of structures is given in [Fischer et al. 2010] for an extension of the μ -calculus, though not in terms of a formal power series over a semiring. Disjunction and conjunction correspond to maximum and minimum, respectively, and temporal as well as fixed-point operators are defined using supremum, infimum, and discount factors. As opposed to our setting, the structures themselves are branching and quantitative (like in probabilistic CTL).

Outline. In Section 2, we recall the classical concept of recognizable formal power series and recall basics about logics. Weighted logics are introduced in Section 3. In Section 4, the class of recognizable series is revisited in the light of these weighted logics, which yields new logical characterizations. In Section 5, we go beyond recognizable series by introducing pebble weighted automata (two-way and one-way), and we show that both formalisms are expressively equivalent to weighted first-order logic augmented with transitive closure. Finally, Section 6 studies some algorithmic questions such as evaluation (the weighted version of the membership problem), and satisfiability. We conclude in Section 7.

2. NOTATION AND BACKGROUND

In this section we set up the notation and we recall some basic results on weighted automata and logics. We refer the reader to [Droste and Gastin 2009; Droste et al. 2009] for details.

Throughout the paper, A denotes a finite alphabet and A^* (respectively, A^+) is the free monoid (respectively, semigroup) over A , i.e., the set of words (respectively, nonempty words). The length of $u \in A^*$ is denoted by $|u|$. If $|u| = n \geq 1$, we usually write $u = u_1 \cdots u_n$ with $u_i \in A$ and we let $\text{pos}(u) = \{1, \dots, n\}$. For $1 \leq i \leq j \leq n$, we denote by $u[i..j]$ the factor $u_i u_{i+1} \cdots u_j$ of u . Finally, we let $A^{\leq k} = \bigcup_{0 \leq i \leq k} A^i$.

2.1. Formal power series

A *semiring* is a structure $(\mathbb{S}, +, \times, 0, 1)$ where $(\mathbb{S}, +, 0)$ is a commutative monoid, $(\mathbb{S}, \times, 1)$ is a monoid, $\times$ distributes over $+$, and 0 is an absorbing element for $\times$. We say that $\mathbb{S}$ is *commutative* if so is $(\mathbb{S}, \times, 1)$.

Example 2.1. We shall refer in the examples to the usual boolean semiring $(\{0, 1\}, \vee, \wedge, 0, 1)$, denoted by $\mathbb{B}$, the semiring $(\mathbb{N}, +, \times, 0, 1)$ of natural numbers, the semiring $(\mathbb{Z}, +, \times, 0, 1)$ of integers, the tropical semiring $(\mathbb{N} \cup \{\infty\}, \min, +, \infty, 0)$, denoted by $\mathbb{T}$, the arctic semiring $(\mathbb{N} \cup \{-\infty\}, \max, +, -\infty, 0)$, denoted by $\mathbb{A}$, and the probabilistic semiring $(\mathbb{R}^+, +, \times, 0, 1)$, denoted by $\mathbb{P}$.

A *formal power series* (or *series*, for short) is a mapping $f : A^+ \rightarrow \mathbb{S}$. The set of formal power series is denoted by $\mathbb{S}\langle\langle A^+ \rangle\rangle$. We denote again by $+$ and $\times$ the pointwise addition and multiplication (called the *Hadamard product*) on $\mathbb{S}\langle\langle A^+ \rangle\rangle$, and by 0 and 1 the constant series with values 0 and 1 , respectively.

2.2. Weighted automata

All automata we consider are finite. A (*one-way*) *weighted automaton* (1WA) over $(\mathbb{S}, +, \times, 0, 1)$ is a tuple $\mathcal{A} = (Q, A, I, F, \Delta, \text{weight})$, where Q is the set of states, A is a finite alphabet, $I \subseteq Q$ is the set of initial states, $F \subseteq Q$ is the set of final states, $\Delta \subseteq Q \times A \times Q$ is the set of transitions, and $\text{weight} : \Delta \rightarrow \mathbb{S}$ assigns a weight to every transition. A *run* on a word $u = u_1 \cdots u_n$ is a sequence of transitions $\rho = q_0 \xrightarrow{u_1} q_1 \xrightarrow{u_2} \cdots \xrightarrow{u_n} q_n$. The *weight* of the run ρ is

$$\text{weight}(\rho) \stackrel{\text{def}}{=} \prod_{i=1}^n \text{weight}(q_{i-1}, u_i, q_i).$$

Such a run ρ is said to be *accepting* if $q_0 \in I$ and $q_n \in F$. The *weight* $\llbracket \mathcal{A} \rrbracket(u)$ of u is the sum of the weights of all accepting runs on u .

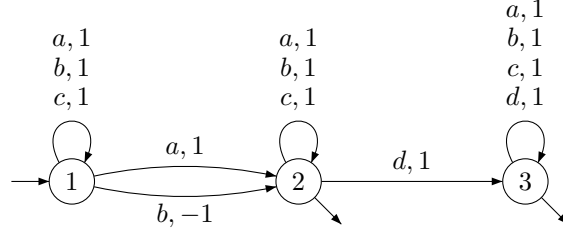


Fig. 1. Automaton for Example 2.3

One may also describe a weighted automaton by matrices: for all $a \in A$, the transition matrix $\mu(a) \in \mathbb{S}^{Q \times Q}$ is defined by $\mu(a)_{q,q'} = \text{weight}(q, a, q')$ for all $q, q' \in Q$ if $(q, a, q') \in \Delta$, and 0 otherwise. Moreover, one defines matrices $\lambda \in \mathbb{S}^{1 \times Q}$ and $\gamma \in \mathbb{S}^{Q \times 1}$ as characteristic matrices of the sets I and F respectively, i.e., for every state $q \in Q$, $\lambda_q = 1$ if $q \in I$, 0 otherwise, and $\gamma_q = 1$ if $q \in F$, 0 otherwise. The mapping $\mu : A \rightarrow \mathbb{S}^{Q \times Q}$ extends uniquely to a morphism $\mu : A^+ \rightarrow \mathbb{S}^{Q \times Q}$ (where $\mathbb{S}^{Q \times Q}$ is endowed with the usual multiplication of matrices). One can then verify that $\llbracket \mathcal{A} \rrbracket(u) = \lambda \times \mu(u) \times \gamma$ (see, e.g., [Berstel and Reutenauer 2010, Prop. 6.1] for more details).

We call $\llbracket \mathcal{A} \rrbracket \in \mathbb{S}\langle\langle A^+ \rangle\rangle$ the *behavior*, or *semantics* of $\mathcal{A}$. A formal power series $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$ is called *recognizable* if it is the behavior of some weighted automaton. We let $\mathbb{S}^{\text{rec}}\langle\langle A^+ \rangle\rangle$ be the collection of all recognizable formal power series.

Example 2.2. In the semiring $\mathbb{N}$, let $\mathcal{A}$ be the automaton with a single state q , both initial and final, and transitions (q, a, q) of weight 2 for all $a \in A$. Then, $\llbracket \mathcal{A} \rrbracket(u) = 2^{|u|}$ for all $u \in A^+$.

Example 2.3. Let $A = \{a, b, c, d\}$. Suppose we want to compute, for a word $u \in A^+$, the difference between the number of a 's and the number of b 's before the first occurrence of d . Over the semiring $\mathbb{Z}$, the automaton $\mathcal{A}$ represented in Figure 1 computes this difference relying on the non-deterministic move from state 1 to state 2.

It is well-known [Berstel and Reutenauer 2010] that $\mathbb{S}^{\text{rec}}\langle\langle A^+ \rangle\rangle$ is stable under addition and, if $\mathbb{S}$ is commutative, also under Hadamard product.

2.3. Monadic second-order logic

Let us fix infinite supplies of first-order variables $\text{Var} = \{x, y, z, t, x_1, x_2, \dots\}$, and of second-order variables $\text{VAR} = \{X, Y, X_1, X_2, \dots\}$. The set $\text{MSO}(A)$ (or simply MSO) of monadic second-order formulas over A is given by the grammar:

$$\varphi ::= P_a(x) \mid x \leq y \mid x \in X \mid \neg \varphi \mid \varphi \vee \varphi \mid \exists x \varphi \mid \exists X \varphi$$

where $a \in A$, $x, y \in \text{Var}$ and $X \in \text{VAR}$. As usual, the set $\text{FO}(A)$ (or simply FO) of first-order formulas over A is the fragment of $\text{MSO}(A)$ without second-order quantifications $\exists X$.

For $\varphi \in \text{MSO}(A)$, let $\text{Free}(\varphi)$ denote the set of free variables of φ . If $\text{Free}(\varphi) = \emptyset$, then φ is called a *sentence*. For a finite set $\mathcal{V} \subseteq \text{Var} \cup \text{VAR}$ and a word $u \in A^+$, a $(\mathcal{V}, u)$ -assignment is a function σ that maps first-order variables in $\mathcal{V}$ to elements of $\text{pos}(u)$ and second-order variables in $\mathcal{V}$ to subsets of $\text{pos}(u)$. For $x \in \text{Var}$ and $i \in \text{pos}(u)$, $\sigma[x \mapsto i]$ denotes the $(\mathcal{V} \cup \{x\}, u)$ -assignment that maps x to i and, otherwise, coincides with σ . For $X \in \text{VAR}$ and $I \subseteq \text{pos}(u)$, the $(\mathcal{V} \cup \{X\}, u)$ -assignment $\sigma[X \mapsto I]$ is defined similarly.

A pair (u, σ) , where σ is a $(\mathcal{V}, u)$ -assignment, can be encoded as a word over the extended alphabet $A_{\mathcal{V}} \stackrel{\text{def}}{=} A \times \{0, 1\}^{\mathcal{V}}$. We write a word $(u_1, \sigma_1) \cdots (u_n, \sigma_n) \in A_{\mathcal{V}}^+$ as (u, σ) where $u = u_1 \cdots u_n$ and $\sigma = \sigma_1 \cdots \sigma_n$. We call (u, σ) *valid* if, for each first-order variable $x \in \mathcal{V}$, the x -row of σ contains exactly one 1. If (u, σ) is valid, then σ encodes the $(\mathcal{V}, u)$ -assignment

that maps a first-order variable $x \in \mathcal{V}$ to the unique position carrying 1 in the x -row, and a second-order variable $X \in \mathcal{V}$ to the set of positions carrying 1 in the X -row.

We fix a finite set $\mathcal{V}$ of variables such that $\text{Free}(\varphi) \subseteq \mathcal{V}$. For every $u \in A^+$ and every $(\mathcal{V}, u)$ -assignment σ , we define $u, \sigma \models \varphi$ by induction over the formula φ , as shown in Table I. Note in particular that the semantics of φ only depends on the restriction of σ to free variables of φ in $\mathcal{V}$.

Table I. Semantics of MSO

$u, \sigma \models P_a(x)$	if $u_{\sigma(x)} = a$
$u, \sigma \models x \leq y$	if $\sigma(x) \leq \sigma(y)$
$u, \sigma \models x \in X$	if $\sigma(x) \in \sigma(X)$
$u, \sigma \models \neg \varphi$	if $u, \sigma \not\models \varphi$
$u, \sigma \models \varphi_1 \vee \varphi_2$	if $u, \sigma \models \varphi_1$ or $u, \sigma \models \varphi_2$
$u, \sigma \models \exists x \varphi$	if there exists $i \in \text{pos}(u)$ such that $u, \sigma[x \mapsto i] \models \varphi$
$u, \sigma \models \exists X \varphi$	if there exists $I \subseteq \text{pos}(u)$ such that $u, \sigma[X \mapsto I] \models \varphi$

We use abbreviations for constants, modulo constraints and comparisons, like for example, $x \leq y + 2$. We use **first** and **last** as abbreviations for the first and last positions of a word. All of these shortcuts can be replaced by suitable FO-formulas, except modulo constraints such as “ $x \equiv_\ell m$ ” with $1 \leq m \leq \ell$, which are MSO-definable.

In the sequel, we let FO+mod be the fragment of MSO consisting of FO augmented with modulo constraints $x \equiv_\ell m$ for constants $1 \leq m \leq \ell$ (since the positions of words start with 1, it is more convenient to compute modulo as a value between 1 and ℓ). The semantics is given by

$$\llbracket x \equiv_\ell m \rrbracket(u, \sigma) = \begin{cases} 1 & \text{if } \sigma(x) \equiv m \pmod{\ell} \\ 0 & \text{otherwise} \end{cases}$$

and it is MSO-definable by

$$x \equiv_\ell m \stackrel{\text{def}}{=} \forall X \left([(x \in X) \wedge (\forall y (y \in X \wedge y > \ell) \implies y - \ell \in X)] \implies m \in X \right).$$

3. WEIGHTED LOGICS

Our aim is to define a denotational model to describe the behavior of weighted automata. This has already been done by Droste and Gastin [Droste and Gastin 2009]: they have introduced weighted logics with syntax close to monadic second-order logic, extending the semantics by using addition and product of a semiring to evaluate disjunctions/existential quantifications, and conjunctions/universal quantifications, respectively. At the price of strong restrictions on the shape of the formulas, they succeeded to prove an expressiveness result (in our formalism, this result is stated in Theorem 3.8).

The syntax of [Droste and Gastin 2009] is purely quantitative, though boolean connectives can be expressed indirectly. As it may be somewhat confusing to interpret purely logical formulas in a weighted manner, we slightly modify the original syntax, by clearly separating the logical and the quantitative parts: our weighted logic consists of a boolean kernel, such as MSO or FO, augmented with quantitative operators (addition, multiplication, sum and product quantifications, and possibly weighted transitive closure). Thus, our language will allow us to test explicitly for boolean monadic second-order properties, and to perform computations. We shall see that both formalisms are equivalent. However, in addition to being more intuitive, the new syntax allows flexibility to study expressiveness. For instance, one can investigate how the underlying boolean logic influences the computational power (see Lemma 3.3).

3.1. General definitions

Definition 3.1 (Weighted logics). Given a class $\mathcal{L}$ of boolean formulas, we denote by $\text{wMSO}(\mathcal{L})$ the class of *weighted monadic second-order logic* defined by

$$\Phi ::= s \mid \varphi \mid \Phi \oplus \Phi \mid \Phi \otimes \Phi \mid \bigoplus_x \Phi \mid \bigotimes_x \Phi \mid \bigoplus_X \Phi \mid \bigotimes_X \Phi$$

where $s \in \mathbb{S}$, $\varphi \in \mathcal{L}$, $x \in \text{Var}$ and $X \in \text{VAR}$. Disabling the sum and product indexed by set variables, we define the class $\text{wFO}(\mathcal{L})$ of *weighted first-order logic* by

$$\Phi ::= s \mid \varphi \mid \Phi \oplus \Phi \mid \Phi \otimes \Phi \mid \bigoplus_x \Phi \mid \bigotimes_x \Phi$$

where $s \in \mathbb{S}$, $\varphi \in \mathcal{L}$ and $x \in \text{Var}$.

We denote again by $\text{Free}(\Phi)$ the set of free variables of a formula $\Phi \in \text{wMSO}(\mathcal{L})$. Let again $\mathcal{V}$ be a finite set of variables such that $\text{Free}(\Phi) \subseteq \mathcal{V}$. The semantics $\llbracket \Phi \rrbracket_{\mathcal{V}} \in \mathbb{S}\langle A_{\mathcal{V}}^+ \rangle$ of Φ wrt. $\mathcal{V}$ is defined as follows: if (u, σ) is not valid, we set $\llbracket \Phi \rrbracket_{\mathcal{V}}(u, \sigma) = 0$. Otherwise $\llbracket \Phi \rrbracket_{\mathcal{V}}$ is given in Table II. Hereby, if $\mathbb{S}$ is not commutative, the products follow the natural order on $\text{pos}(u)$ and the lexicographic order on the power set $\{0, 1\}^{\text{pos}(u)}$.

Table II. Semantics of $\text{wMSO}(\mathcal{L})$

$$\begin{aligned} \llbracket s \rrbracket(u, \sigma) &= s & \llbracket \varphi \rrbracket(u, \sigma) &= \begin{cases} 1 & \text{if } u, \sigma \models \varphi \\ 0 & \text{otherwise} \end{cases} \\ \llbracket \Phi_1 \oplus \Phi_2 \rrbracket(u, \sigma) &= \llbracket \Phi_1 \rrbracket(u, \sigma) + \llbracket \Phi_2 \rrbracket(u, \sigma) & \llbracket \Phi_1 \otimes \Phi_2 \rrbracket(u, \sigma) &= \llbracket \Phi_1 \rrbracket(u, \sigma) \times \llbracket \Phi_2 \rrbracket(u, \sigma) \\ \llbracket \bigoplus_x \Phi \rrbracket(u, \sigma) &= \sum_{i \in \text{pos}(u)} \llbracket \Phi \rrbracket(u, \sigma[x \mapsto i]) & \llbracket \bigotimes_x \Phi \rrbracket(u, \sigma) &= \prod_{i \in \text{pos}(u)} \llbracket \Phi \rrbracket(u, \sigma[x \mapsto i]) \\ \llbracket \bigoplus_X \Phi \rrbracket(u, \sigma) &= \sum_{I \subseteq \text{pos}(u)} \llbracket \Phi \rrbracket(u, \sigma[X \mapsto I]) & \llbracket \bigotimes_X \Phi \rrbracket(u, \sigma) &= \prod_{I \subseteq \text{pos}(u)} \llbracket \Phi \rrbracket(u, \sigma[X \mapsto I]) \end{aligned}$$

Example 3.2. In the boolean semiring $\mathbb{B}$, recognizable and $\text{wMSO}(\text{MSO})$ -definable series coincide. In contrast, for the natural semiring $\mathbb{N}$, the definition yields $\llbracket \bigotimes_x \bigotimes_y 2 \rrbracket(u) = 2^{|u|^2}$, which is not recognizable [Droste and Gastin 2009]. Indeed, let $\mathcal{A} = (Q, A, I, F, \Delta, \text{weight})$. Then, $\llbracket \mathcal{A} \rrbracket(u) = O((M|Q|)^{|u|+1})$ for $M = \max(\text{weight}(\Delta))$, since there are $O(|Q|^{|u|+1})$ runs on u , each of weight $O(M^{|u|})$. Also observe that the behavior of the automaton of Example 2.2 is $\llbracket \bigotimes_y 2 \rrbracket$. Therefore, recognizable series are not stable under first-order product $\bigotimes_x$.

We denote by AP the class of atomic propositions, i.e., that contains only formulas $P_a(x)$, $x \leq y$, $x \in X$ and their negations. As explained at the beginning of the section, the logic $\text{wMSO}(\mathbb{S})$ (respectively, $\text{wFO}(\mathbb{S})$) introduced in [Droste and Gastin 2009] is exactly the class of formulas $\text{wMSO}(\text{AP})$ (respectively, $\text{wFO}(\text{AP})$). Transposed into our syntax, the transformation described in their Definition 4.3 and proved in Lemma 4.4, permits to state:

LEMMA 3.3. *For all formulas $\varphi \in \text{MSO}$ (respectively, $\varphi \in \text{FO}$), there exists a formula $\Phi \in \text{wMSO}(\text{AP})$ (respectively, $\Phi \in \text{wFO}(\text{AP})$), with the same set of free variables, such that $\llbracket \varphi \rrbracket = \llbracket \Phi \rrbracket$.*

Hence, we can always replace a boolean FO or MSO formula by its $\text{wFO}(\text{AP})$ or $\text{wMSO}(\text{AP})$ equivalent and we obtain:

COROLLARY 3.4. *Let $\mathbb{S}$ be a (possibly non-commutative) semiring and $f \in \mathbb{S}\langle A^+ \rangle$. Then,*

- *f is $\text{wMSO}(\text{MSO})$ -definable if and only if f is $\text{wMSO}(\text{AP})$ -definable.*

— f is $\text{wFO}(\text{FO})$ -definable if and only if f is $\text{wFO}(\text{AP})$ -definable.

For a boolean formula $\varphi \in \mathcal{L}$ and a formula $\Phi \in \text{wMSO}(\mathcal{L})$, we define the macro

$$\varphi \ominus \Phi \stackrel{\text{def}}{=} \neg\varphi \oplus (\varphi \otimes \Phi)$$

as a natural generalization of the boolean implication ($\varphi \rightarrow \varphi'$ for $\varphi, \varphi' \in \mathcal{L}$): its semantics is $\llbracket \Phi \rrbracket$ if φ holds, and 1 otherwise.

Example 3.5. The series recognized by the weighted automaton in Example 2.3 can also be described by the following formula in $\text{wFO}(\text{FO})$:

$$\bigoplus_y \left[(\neg \exists x (x \leq y \wedge P_d(x))) \otimes (P_a(y) \oplus (P_b(y) \otimes (-1))) \right].$$

Example 3.6. Consider now a machine with one counter that can take non-negative values, being either incremented, decremented or zero-tested on each step. We encode the behavior of such a machine by recording the string of instructions applied on the counter, letters Inc , Dec and $ZTest$ stand for increment, decrement and zero-test respectively. We will show in Theorem 6.2 that satisfiability of a formula in the logic $\text{wFO}(\text{FO})$ over the semiring $\mathbb{Z}$ is undecidable, by reducing the problem of emptiness of a two-counter machine. In order to prepare this theorem, we design formulas that map a word $u \in \{Inc, Dec, ZTest\}^+$ to weight 0 when u encodes a non-valid behavior of a non-negative counter.

First, we want to make sure that the counter always stays non-negative. For this, considering a word $u \in \{Inc, Dec, ZTest\}^+$, and assuming that the counter starts with value 0, the value¹

$$\prod_{i \in \text{pos}(u) \mid u_i = Dec} (|u[1..i-1]|_{Inc} - |u[1..i-1]|_{Dec})$$

is 0 if and only if the counter becomes negative along the sequence of instructions encoded by u , since at the first position i where it becomes negative, we have $|u[1..i-1]|_{Inc} = |u[1..i-1]|_{Dec}$. By using a simpler version of the formula described in Example 3.5, we can compute this value with the following formula of $\text{wFO}(\text{AP})$:

$$\bigotimes_x \left[P_{Dec}(x) \ominus \bigoplus_y \left[y < x \otimes (P_{Inc}(y) \oplus (P_{Dec}(y) \otimes (-1))) \right] \right].$$

Next, assuming that the counter always stays non-negative, we want to check that a zero-test is used only when the counter is indeed 0. For a word $u \in \{Inc, Dec, ZTest\}^+$, let us consider the value

$$\prod_{i \in \text{pos}(u) \mid u_i = ZTest} \prod_{1 \leq k < i} (|u[1..i-1]|_{Inc} - |u[1..i-1]|_{Dec} - k).$$

Note that, assuming that the counter is 0 initially and always stays non-negative, this value is 0 if and only if some zero-test occurs when the counter is not zero. Note that this series can be defined by the following formula in $\text{wFO}(\text{AP})$:

$$\bigotimes_x \left[P_{ZTest}(x) \ominus \bigotimes_z \left[z < x \ominus \bigoplus_y \left[(y \leq z \otimes (-1)) \oplus (y < x \otimes (P_{Inc}(y) \oplus (P_{Dec}(y) \otimes (-1)))) \right] \right] \right]$$

3.2. Previous expressiveness result

For $\mathcal{L} \subseteq \text{MSO}$ closed under $\vee$, $\wedge$ and $\neg$, an $\mathcal{L}$ -step formula is a formula obtained from the grammar

$$\Phi ::= s \mid \varphi \mid \Phi \oplus \Phi \mid \Phi \otimes \Phi, \quad \text{with } s \in \mathbb{S} \text{ and } \varphi \in \mathcal{L}. \quad (1)$$

The following lemma shows in particular that an $\mathcal{L}$ -step formula assumes a finite number of values, each of which corresponds to an $\mathcal{L}$ -definable language.

¹Here, and in the following, $|u|_a$ stands for the number of occurrences of letter a in word u .

LEMMA 3.7. *For every $\mathcal{L}$ -step formula Φ , one can construct an equivalent formula $\Psi = \bigoplus_{i \in I} (\psi_i \otimes s_i)$ with I finite, $\psi_i \in \mathcal{L}$ and $s_i \in \mathbb{S}$. More precisely, $\text{Free}(\Phi) = \text{Free}(\Psi)$ and for every $\mathcal{V}$ containing $\text{Free}(\Phi)$, $\llbracket \Phi \rrbracket_{\mathcal{V}}(u, \sigma) = \llbracket \Psi \rrbracket_{\mathcal{V}}(u, \sigma)$ for all $(u, \sigma) \in A_{\mathcal{V}}^+$.*

PROOF. Call $\varphi_1, \dots, \varphi_p$ the $\mathcal{L}$ -formulas occurring in the expression of Φ given by the grammar (1). For $I \subseteq \{1, \dots, p\}$, let Φ_I be the formula obtained by replacing in Φ each φ_i by 1 if $i \in I$ and by 0 otherwise, so that Φ_I has no free variable and $\llbracket \Phi_I \rrbracket$ is a constant $s_I \in \mathbb{S}$. Let $\psi_I = \bigwedge_{i \in I} \varphi_i \wedge \bigwedge_{i \notin I} \neg \varphi_i$, which is an $\mathcal{L}$ -formula, since $\mathcal{L}$ is closed under $\wedge$ and $\neg$. Let $\Psi = \bigoplus_I (\psi_I \otimes s_I)$. Clearly, $\text{Free}(\Phi) = \text{Free}(\Psi)$.

Let $\mathcal{V} \supseteq \text{Free}(\Phi)$. Fix a word u and a $(\mathcal{V}, u)$ -assignment σ . Let $J = \{i \mid (u, \sigma) \models \varphi_i\}$, so that $(u, \sigma) \models \psi_J$ and $(u, \sigma) \not\models \psi_I$ for $I \neq J$. Therefore, $\llbracket \Psi \rrbracket_{\mathcal{V}}(u, \sigma) = s_J = \llbracket \Phi_J \rrbracket_{\mathcal{V}}(u, \sigma) = \llbracket \Phi \rrbracket_{\mathcal{V}}(u, \sigma)$, where the last equality comes from the definition of J . $\square$

From now on, we freely use Lemma 3.7, using the special form it provides for $\mathcal{L}$ -step formulas. All MSO-step formulas are clearly recognizable. By [Droste and Gastin 2009], $\bigotimes_x \Phi$ is recognizable for any MSO-step formula Φ . A restricted fragment sREMSO of wMSO(AP) (called the syntactically restricted formulas), was defined in [Droste and Gastin 2009]. Essentially, sREMSO consists of wMSO(MSO) formulas of the form $\bigoplus_{X_1} \dots \bigoplus_{X_n} \Phi$ where Φ is a wFO(MSO) formula such that if it contains $\Psi \otimes \Psi'$ as a subformula then the values in $\mathbb{S}$ appearing in Ψ and Ψ' commute element-wise, and such that the use of first-order product $\bigotimes_x$ is restricted to MSO-step formulas. Note that if the semiring is commutative, the requirement over the product $\Psi \otimes \Psi'$ is trivially verified. A consequence of their work is the following statement.

THEOREM 3.8 ([DROSTE AND GASTIN 2009]). *A formal power series is recognizable if and only if it is definable in sREMSO.*

3.3. Weighted transitive closure

We now define other fragments of wMSO(MSO), which also carry enough expressiveness to generate all recognizable series. Contrary to sREMSO that allows second-order quantitative operators, and finally restricts them, we rather extend the fragment wFO(MSO), containing only first-order quantitative operators, with a weighted equivalent of the boolean transitive closure.

We first define a very general weighted transitive closure operator before presenting some restrictions over it that will be useful in the sequel.

Weighted transitive closure. For a formula $\Phi(x, y)$ with at least two free variables x and y , we introduce a weighted transitive closure operator $[\text{TC}_{x,y}\Phi](x', y')$ having as free variables the fresh variables x' and y' , and all the free variables of Φ except x and y . Its semantics is defined by

$$\llbracket [\text{TC}_{x,y}\Phi](x', y') \rrbracket(u, \sigma) = \sum_{\sigma(x')=i_0, i_1, \dots, i_m=\sigma(y')} \prod_{0 \leq k \leq m-1} \llbracket \Phi \rrbracket(u, \sigma[x \mapsto i_k, y \mapsto i_{k+1}]) \quad (2)$$

where the sum runs over all $m > 0$ and all sequences $(i_k)_{0 \leq k \leq m}$ of positions of the word u with $i_0 = \sigma(x')$ and $i_m = \sigma(y')$ such that either $m = 1$, or $m > 1$ and all positions of the sequence are pairwise distinct. This sum is finite as the word has finitely many positions. Notice that if $\sigma(x') = \sigma(y') = i$, then the semantics is $\llbracket \Phi \rrbracket(u, \sigma[x \mapsto i, y \mapsto i])$.

To ease notation, we now write $\llbracket \Phi(x, y) \rrbracket(u, i, j)$ instead of $\llbracket \Phi(x, y) \rrbracket(u, [x \mapsto i, y \mapsto j])$.

Example 3.9. Let $\Psi = \text{TC}_{x,y} 1$ over $\mathbb{N}$. Then for a word u of length n , we have $\llbracket \Psi \rrbracket(u, 1, n) = \sum_{m=0}^{n-2} m! \binom{n-2}{m}$, since the sum in (2) ranges over all sequences of pairwise distinct positions in $\{1, \dots, n\}$ starting in 1 and ending in n .

Ordered weighted transitive closure. Since later we consider one-way weighted automata, it is natural to restrict this operator to a left-to-right version. Hence, we introduce the operator $\text{TC}_{x,y}^<\Phi$ whose semantics consists in restricting the sum of (2) to *non-decreasing* sequences $(i_k)_{0 \leq k \leq m}$ of positions in the word. Equivalently, we can define it by

$$\text{TC}_{x,y}^<\Phi \stackrel{\text{def}}{=} \text{TC}_{x,y}(x \leq y \otimes \Phi).$$

Intuitively, the $\text{TC}_{x,y}^<$ operator generalizes the forward transitive closure operator of the boolean case: a formula $\varphi(x, y)$ with two free variables defines a binary relation on positions of a word u , namely $\{(i, j) \mid (i \leq j) \wedge u, i, j \models \varphi\}$. The relation defined by $\text{TC}_{x,y}^<\varphi$ is the transitive closure of this relation.

Throughout this paper, we will mostly use ordered transitive closure. For this reason, we define recursively the m -th power of a formula Φ , with this ordered restriction

$$\begin{aligned} \Phi^1(x, y) &\stackrel{\text{def}}{=} (x \leq y) \otimes \Phi(x, y), \\ \Phi^{m+1}(x, y) &\stackrel{\text{def}}{=} \bigoplus_z [(x < z < y) \otimes \Phi(x, z) \otimes \Phi^m(z, y)], \quad \text{for } m \geq 1. \end{aligned}$$

The left-to-right restriction of the transitive closure permits finally to give a third equivalent definition, *a priori* simpler to use:

$$\llbracket \text{TC}_{x,y}^<\Phi \rrbracket = \sum_{m \geq 1} \llbracket \Phi^m(x, y) \rrbracket.$$

This infinite sum is well-defined: $\llbracket \Phi^m(x, y) \rrbracket(u, \sigma) = 0$ if $m \geq |u|$, i.e., on each pair (u, σ) , only finitely many terms assume a nonzero value.

Bounded weighted transitive closure. We also introduce a *bounded* weighted transitive closure operation $\text{TC}_{x,y}^N$ for each integer $N > 0$. Its semantics consists in restricting the sequences of (2) so that $|i_{k+1} - i_k| \leq N$ for every $k \in \{0, \dots, m-1\}$. Equivalently,

$$\text{TC}_{x,y}^N \Phi \stackrel{\text{def}}{=} \text{TC}_{x,y}(|y - x| \leq N \otimes \Phi).$$

Bounded ordered weighted transitive closure. Finally, we can combine both restrictions and introduce a *bounded ordered* weighted transitive closure operation. For each integer $N > 0$, the $\text{TC}_{x,y}^{N,<}$ operator is given by

$$\text{TC}_{x,y}^{N,<} \Phi = \text{TC}_{x,y}((x \leq y \leq x + N) \otimes \Phi). \quad (3)$$

Definition 3.10. Given a class $\mathcal{L}$ of boolean formulas, we denote by $\text{wFOTC}(\mathcal{L})$ the class of weighted formulas defined by

$$\Phi ::= s \mid \varphi \mid \Phi \oplus \Phi \mid \Phi \otimes \Phi \mid \bigoplus_x \Phi \mid \bigotimes_x \Phi \mid \text{TC}_{x,y} \Phi$$

with $s \in \mathbb{S}$, $\varphi \in \mathcal{L}$, $x, y \in \text{Var}$. A series $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$ is said to be $\text{wFOTC}(\mathcal{L})$ -definable if there is a formula $\Phi \in \text{wFOTC}(\mathcal{L})$ such that $\llbracket \Phi \rrbracket = f$. We define similar notions for the restricted operators $\text{TC}^<$, TC^N and $\text{TC}^{N,<}$, denoting them $\text{wFOTC}^<(\mathcal{L})$, $\text{wFOTC}^b(\mathcal{L})$ and $\text{wFOTC}^{b,<}(\mathcal{L})$, respectively: in particular, for the bounded restrictions, we allow all constants N in the transitive closure operators.

Notice that if $\Phi(x)$ is a formula from $\text{wFOTC}(\mathcal{L})$ with a free variable x , then, for a fixed integer k , the formula $\bigoplus_y [(y = x + k) \otimes \Phi(y)]$ is again a formula of the same fragment, that we will denote $\Phi(x + k)$ in the following.

Example 3.11. Let $\Phi(x, y) \stackrel{\text{def}}{=} \bigotimes_z 2$ over $\mathbb{N}$. Let $u = u_1 \cdots u_n$ with $n \geq 2$. We have $\llbracket \text{TC}_{x,y}^{1,<} \Phi \rrbracket(u, 1, n) = \prod_{i=1}^{n-1} \llbracket \Phi \rrbracket(u, i, i+1)$ due to the bounded and ordered weighted transitive closure. Now, $\llbracket \Phi \rrbracket(u, i, i+1) = 2^{|u|}$ if $i \geq 1$, so $\llbracket \text{TC}_{x,y}^{1,<} \Phi \rrbracket(u, 1, n) = 2^{|u|(|u|-1)}$. Noticing that the series $u \mapsto 2^{|u|}$ is recognizable by a weighted automaton, but not the series $u \mapsto 2^{|u|(|u|-1)}$

(similarly to Example 3.2), this example shows in particular that the class of recognizable series is not closed under application of bounded ordered weighted transitive closures.

Example 3.12. Consider the formula $\Phi = \text{TC}_{x,y}^{2,<}(2 \otimes y = x + 2)$ over A and $\mathbb{N}$. Then

$$\llbracket \Phi \rrbracket(u, 1, |u|) = \begin{cases} 2^n & \text{if } |u| = 2n + 1 \text{ with } n \geq 1 \\ 0 & \text{otherwise.} \end{cases}$$

Notice that the support of $\llbracket \Phi \rrbracket$ (i.e., the language of all words mapped to nonzero values) is not FO-definable. Therefore, $\llbracket \Phi \rrbracket$ is not wFO(FO)-definable, since otherwise, we would have $\llbracket \Phi \rrbracket = \llbracket \Psi \rrbracket$ for a wFO(FO)-formula Ψ , and the FO formula obtained from Ψ by changing all nonzero constants of $\mathbb{N}$ to 1 and replacing $\oplus$, $\otimes$, $\bigoplus_x$ and $\bigotimes_x$ by $\vee$, $\wedge$, $\exists x$ and $\forall x$ respectively would define the support of $\llbracket \Phi \rrbracket$ (this holds because $\mathbb{N}$ is a positive semiring).

Example 3.13. The *modulo* predicate can easily be expressed in $\text{wFOTC}^{b,<}(\text{FO})$. For $1 \leq m \leq \ell$, the predicate is expressed by

$$x \equiv_\ell m \stackrel{\text{def}}{=} (x = m) \oplus \left[\text{TC}_{x',y'}^{\ell,<}(y' = x' + \ell) \right] (m, x)$$

while the binary relation can be expressed with

$$x \equiv_\ell y \stackrel{\text{def}}{=} (y = x) \oplus \left[\text{TC}_{x',y'}^{\ell,<}(y' = x' + \ell) \right] (x, y) \oplus \left[\text{TC}_{x',y'}^{\ell,<}(y' = x' + \ell) \right] (y, x).$$

4. EXPRESSIVENESS OF WEIGHTED LOGICS

We now consider syntactical restrictions of $\text{wFOTC}^{<}(\text{FO})$ and $\text{wFOTC}^{b,<}(\text{FO})$, inspired by normal form formulas of [Neven and Schwentick 2000] where only one “external” weighted transitive closure operator is allowed.

In the following, sentences like “ f is definable by a formula $\text{TC}_{x,y}\Psi$ ” means that for every word $u \in A^+$, we have

$$f(u) = \llbracket \text{TC}_{x,y}\Psi \rrbracket(u, 1, |u|)$$

The following result characterizes the expressive power of weighted automata.

THEOREM 4.1. *Let $\mathbb{S}$ be a (possibly non-commutative) semiring and $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$. The following assertions are equivalent over $\mathbb{S}$ and A .*

- (1) f is recognizable.
- (2) f is definable by a formula $\text{TC}_{x,y}^{N,<}\Phi$ with Φ (FO+mod)-step.
- (3) f is definable by a formula $\text{TC}_{x,y}^{N,<}\Phi$ with Φ MSO-step.
- (4) f is definable by a formula $\text{TC}_{x,y}^{<}\Phi$ with Φ (FO+mod)-step.
- (5) f is definable by a formula $\text{TC}_{x,y}^{<}\Phi$ with Φ MSO-step.
- (6) f is definable by a formula $\bigoplus_X \bigotimes_x \Phi(x, X)$ with Φ FO-step.
- (7) f is definable by a formula $\bigoplus_X \bigotimes_x \Phi(x, X)$ with Φ MSO-step.

PROOF. The implications $2 \Rightarrow 3$, $4 \Rightarrow 5$ and $6 \Rightarrow 7$ are clear since $\text{FO+mod} \subseteq \text{MSO}$, and $7 \Rightarrow 1$ follows from Theorem 3.8 because formulas described in 7 are included in sREMSO: indeed, because of Lemma 3.7, every product $\Psi \otimes \Psi'$ in Φ is such that $\Psi \in \text{MSO}$ and $\Psi' = s \in \mathbb{S}$, henceforth the property on elementwise commutation is trivially verified. The implications $2 \Rightarrow 4$ and $3 \Rightarrow 5$ are also obvious since $\text{TC}^{N,<}$ can be defined with $\text{TC}^{<}$. We prove $1 \Rightarrow 2$, $1 \Rightarrow 6$, and $5 \Rightarrow 7$.

For the first two implications, let us fix a weighted automaton $\mathcal{A} = (Q, A, I, F, \Delta, \text{weight})$ with $Q = \{1, \dots, m\}$. Recall that μ denotes the morphism from A^+ to $\mathbb{S}^{Q \times Q}$ as defined in Section 2.2. The general idea behind the constructions we will present is that the semantics $\llbracket \mathcal{A} \rrbracket(u)$ can be computed by splitting the word u in small slices $u_1, \dots, u_k$ by $\llbracket \mathcal{A} \rrbracket(u) =$

$\lambda \times \mu(u_1) \times \cdots \times \mu(u_k) \times \gamma$. As it is a priori not possible to compute a matrix of weight in the logic without changing the semiring, we must moreover partition the accepting runs considering the sequence of states they encounter at every positions between slice u_i and u_{i+1} . Each state must then be encoded into the word so that a formula can find it to check the validity of the run and its weight: to do so, we consider portions $u_1, \dots, u_k$ to be of size m roughly, and position $q \in Q$ of u_i encodes that state q were encountered before reading u_i in the current run. It is then necessary to compute the weight of runs over a slice. For $v \in A^+$ and $p, q \in Q$, we use an FO-step formula $\Xi_{p,q}^v(x)$ to compute the weight of a factor starting at position $x - p + 1$, labeled by the word v , when $\mathcal{A}$ goes from p to q :

$$\Xi_{p,q}^v(x) \stackrel{\text{def}}{=} \mu(v)_{p,q} \otimes \bigwedge_{1 \leq k \leq |v|} P_{v_k}(x - p + k).$$

We easily see that for every word u , and every positions $i, j \in \text{pos}(u)$ with $i \leq j$ and $i + p - 1 \leq |u|$,

$$\llbracket \Xi_{p,q}^{u[i..j]}(x) \rrbracket(u, i + p - 1) = \mu(u[i..j])_{p,q}. \quad (4)$$

Let us show $1 \Rightarrow 2$. We construct an (FO+mod)-step formula $\Phi_{\text{long}}(x, y)$, such that $\llbracket \mathcal{A} \rrbracket(u) = \llbracket \text{TC}_{x,y}^{2m, <} \Phi_{\text{long}} \rrbracket(u, 1, |u|)$ for every word u *sufficiently long* (recall that m is the number of states of $\mathcal{A}$). As already mentioned, the idea, inspired by [Thomas 1982], consists in making the transitive closure operator pick positions $z_\ell = \ell m + q_\ell$, with $1 \leq q_\ell \leq m$, for successive values of ℓ , to encode runs of $\mathcal{A}$ going through state q_ℓ just before reading the letter at position $\ell m + 1$. Consider slices $[\ell m + 1, (\ell + 1)m]$ of positions in the word where we evaluate the formula (the last slice might be incomplete). Each position x is located in exactly one such slice, and the state p encoded by such a position is given by the FO+mod formula $x \equiv_m p$. Moreover, the first position of the slice in which a position x is located can be computed by $x - p + 1$, if $x \equiv_m p$. Since the transitive closure starts from the first position of the word, to make this still work for $\ell = 0$, let us assume without loss of generality that $I = \{1\}$.

Our $\text{TC}^{N, <}$ -formula picks positions x and y marked $\bullet$ in Figure 2, and computes the weight of the factor of length m between positions $x - p + 1$ and $x - p + m$ (included), assuming states $x \equiv_m p$ and $y \equiv_m q$ just before and after these positions respectively.

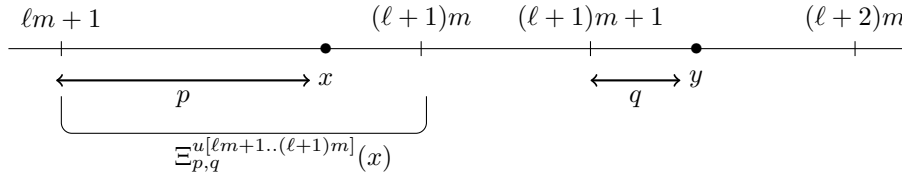


Fig. 2. Positions picked by the $\text{TC}^{N, <}$ -formula

The formula Φ_{long} distinguishes the cases where x is located far or near from the last position. More precisely, in the following expression of Φ_{long} , the first sum corresponds to the case where an entire slice is still available after the slice of x (as in Figure 2), while the second sum captures the case where x is located in the last complete slice.

$$\begin{aligned} \Phi_{\text{long}}(x, y) = & \bigoplus_{p, q \in Q, v \in A^m} (x - p + 2m < \text{last} \wedge x \equiv_m p \wedge y - q = x - p + m) \otimes \Xi_{p,q}^v(x) \\ & \oplus \bigoplus_{\substack{p \in Q, q_f \in F \\ m < d \leq 2m, v \in A^d}} (x - p + d = \text{last} \wedge x \equiv_m p \wedge y = \text{last}) \otimes \Xi_{p,q_f}^v(x). \end{aligned}$$

Using (4), this definition implies that, for every word u and all positions $\ell m + p$, $\ell' m + q$ in u , with $p, q \in Q$:

$$\llbracket \Phi_{\text{long}} \rrbracket(u, \ell m + p, \ell' m + q) = \begin{cases} \mu(u[\ell m + 1.. \ell' m])_{p,q} & \text{if } \ell' = \ell + 1 \wedge (\ell + 2)m < |u|, \\ \sum_{q_f \in F} \mu(u[\ell m + 1.. |u|])_{p,q_f} & \text{if } \ell' = \ell + 1 \wedge \ell' m + q = |u|, \\ 0 & \text{otherwise.} \end{cases} \quad (5)$$

Notice that in the second case, we have $(\ell + 2)m = \ell' m + m \geq \ell' m + q = |u|$. This ensures that the first two cases are disjoint. Now, by definition of the $\text{TC}_{x,y}^{2m,<}$ operator,

$$\llbracket \text{TC}_{x,y}^{2m,<} \Phi_{\text{long}}(x, y) \rrbracket(u, 1, |u|) = \sum_{k \geq 1} \llbracket \Phi_{\text{long}}^k(x, y) \rrbracket(u, 1, |u|).$$

Observe that Φ_{long} syntactically ensures the bounded and ordered condition of the transitive closure operator.

Suppose now that $|u| \geq m + 1$. We show that there is at most one nonzero value in this sum: $\llbracket \Phi_{\text{long}}^k(x, y) \rrbracket(u, 1, |u|)$ with $k = \lfloor \frac{|u|-1}{m} \rfloor$. So let k be the largest integer ℓ such that $\ell m < |u|$.

Let $1 = i_0 < \dots < i_h = |u|$ be a sequence of positions chosen in (2) such that $\llbracket \Phi_{\text{long}}(x, y) \rrbracket(u, i_\ell, i_{\ell+1}) \neq 0$ for all $\ell \in \{0, \dots, h-1\}$. For $\ell \leq h-1$, we denote by $q_\ell \in Q$ the state such that $i_\ell \equiv_m q_\ell$. The first case of (5) must apply for $0 \leq \ell \leq h-2$, and it yields by induction $i_\ell = \ell m + q_\ell$ if $0 \leq \ell \leq h-1$. The second case of (5) applied with $\ell = h-1$ yields $|u| \leq (h+1)m$, whence $h = k = \lfloor \frac{|u|-1}{m} \rfloor$.

In particular, if $|u| \leq m$, then $\llbracket \text{TC}_{x,y}^{2m,<} \Phi_{\text{long}}(x, y) \rrbracket(u, 1, |u|) = 0$. If $|u| > m$, we deduce that $\llbracket \text{TC}_{x,y}^{2m,<} \Phi_{\text{long}}(x, y) \rrbracket(u, 1, |u|) = \llbracket \Phi_{\text{long}}^k(x, y) \rrbracket(u, 1, |u|)$ for $k = \lfloor \frac{|u|-1}{m} \rfloor \geq 1$. The assignment $1 = i_0 < i_1 < \dots < i_k = |u|$ is uniquely defined by the sequence $\bar{q} = (q_1, \dots, q_{k-1}) \in Q^{k-1}$ (note that if $k = 1$, then $\bar{q}$ is empty). The possible choices of this tuple induce the sum in the following evaluation of $\llbracket \Phi_{\text{long}}^k(x, y) \rrbracket(u, 1, |u|)$, where we omit the variable names (x, y) in the right hand side. Recall that $q_0 = 1$ is the unique initial state giving a nonzero value.

$$\begin{aligned} \llbracket \text{TC}_{x,y}^{2m,<} \Phi_{\text{long}}(x, y) \rrbracket(u, 1, |u|) &= \llbracket \Phi_{\text{long}}^k \rrbracket(u, 1, |u|) \\ &= \sum_{\bar{q} \in Q^{k-1}} \left(\left[\prod_{\ell=1}^{k-1} \llbracket \Phi_{\text{long}} \rrbracket(u, (\ell-1)m + q_{\ell-1}, \ell m + q_\ell) \right] \times \llbracket \Phi_{\text{long}} \rrbracket(u, (k-1)m + q_{k-1}, |u|) \right) \\ &= \sum_{\bar{q} \in Q^{k-1}} \left(\left[\prod_{\ell=1}^{k-1} \mu(u[(\ell-1)m + 1.. \ell m])_{q_{\ell-1}, q_\ell} \right] \times \sum_{q_f \in F} \mu(u[(k-1)m + 1.. |u|])_{q_{k-1}, q_f} \right) \\ &= \sum_{q_f \in F} \mu(u)_{q_0, q_f} = \llbracket \mathcal{A} \rrbracket(u). \end{aligned}$$

Finally, if $|u| \leq m$, we can design an FO-step formula Φ_{small} dealing with small words:

$$\Phi_{\text{small}} \stackrel{\text{def}}{=} (x = \text{first} \wedge y = \text{last}) \otimes \bigoplus_{\substack{1 \leq d \leq m \\ v \in A^d, q_f \in F}} (\text{last} = d) \otimes \Xi_{q_0, q_f}^v(\text{first}).$$

Only the term for $v = u$ yields a nonzero value. Hence, $\text{TC}_{x,y}^{2m,<}(\Phi_{\text{small}} \oplus \Phi_{\text{long}})$ computes $\llbracket \mathcal{A} \rrbracket$, as required.

Let us now outline the proof of $1 \Rightarrow 6$, which relies on a similar technique. We use the monadic second-order variable X to partition the runs along the portions $u = u_1 \dots u_k$

considering the states in-between each portion. The set X will contain for all i : x_i , the first position of u_i and t_i , the position in u_i used to encode the state of Q reached before reading u_i . To distinguish x_i 's and t_i 's, we consider portions of size $2m+1$, so that between x_i and t_i the distance is no more than m , whereas the distance between t_i and x_{i+1} is at least m .

More formally, we encode runs of $\mathcal{A}$ by a formula $\bigoplus_X \bigotimes_x \Phi(x, X)$ with Φ an FO-step formula, enforcing X to represent positions $x_0 < t_0 < x_1 < t_1 < \dots$ with $x_\ell = (2m+1)\ell + 1$, where the distance $t_\ell - x_\ell$ encodes the state of $\mathcal{A}$ reached just before position x_ℓ . For instance, since we assume as above that the only initial state yielding a nonzero value is $q_0 = 1$, we enforce $x_0 = 1$ and $t_0 = 2$, so that $t_0 - x_0 = 1$ encodes that state. We enforce $1 \leq t_\ell - x_\ell \leq m$ (so that $t_\ell - x_\ell \in Q$) but we make a gap between t_ℓ and $x_{\ell+1}$, namely $x_{\ell+1} - t_\ell > m$.

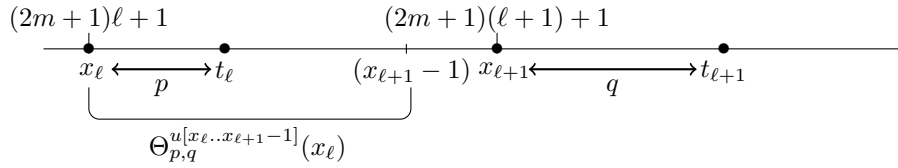


Fig. 3. Positions represented by X , marked $\bullet$

We use shortcuts like $\{z, t\} \subseteq X$, or $X \cap]z, t] = \emptyset$, where $]z, t] = \{z' \mid z < z' \leq t\}$, which are easily FO-definable. Let us define

$$\begin{aligned} \Phi(x, X) = & (\text{last} > m \wedge \{1, 2\} \subseteq X) \otimes (\Phi_{\text{far}}(x, X) \oplus \Phi_{\text{near}}(x, X)) \\ & \oplus (\text{last} \leq m \wedge X = \emptyset) \otimes [x = \text{first} \ominus \bigoplus_{\substack{1 \leq d \leq m \\ v \in A^d, q_f \in F}} (\text{last} = d \otimes \Theta_{q_0, q_f}^v(x))]. \end{aligned}$$

with $\Theta_{p,q}^v(x)$ a slightly modified version of $\Xi_{p,q}^v(x)$:

$$\Theta_{p,q}^v(x) \stackrel{\text{def}}{=} \mu(v)_{p,q} \otimes \bigwedge_{1 \leq k \leq |v|} P_{v_k}(x + k - 1)$$

verifying, for every word u , and every positions $i, j \in \text{pos}(u)$ with $i \leq j$,

$$\llbracket \Theta_{p,q}^{u[i..j]}(x) \rrbracket(u, i) = \mu(u[i..j])_{p,q}. \quad (6)$$

The outermost sum in Φ distinguishes the cases of long or short words, while the sum $\Phi_{\text{far}}(x, X) \oplus \Phi_{\text{near}}(x, X)$ discriminates between positions $x \in X$ located far/near from the last position. For the case of short words, the product resulting from the quantification $\bigotimes_x$ has value $\llbracket \mathcal{A} \rrbracket(u)$ thanks to the premise $x = \text{first}$ of the implication $\ominus$. Moreover, the sum resulting from $\bigoplus_X$ has a single nonzero value thanks to $X = \emptyset$. Therefore, the overall semantics is indeed $\llbracket \mathcal{A} \rrbracket(u)$. We define Φ_{far} as

$$\begin{aligned} \Phi_{\text{far}}(x, X) = & (x + 3m + 1 \leq \text{last}) \otimes ((x \in X \wedge X \cap]x, x + m] \neq \emptyset) \ominus \\ & \bigoplus_{\substack{p, q \in Q \\ v = v_1 \dots v_{2m+1}}} X \cap]x, x + 3m + 1] = \{x + p, x + 2m + 1, x + 2m + 1 + q\} \otimes \Theta_{p,q}^v(x). \end{aligned}$$

Note that p and q are uniquely defined by the set X . To produce a nonzero value, we must also have $x + 3m + 1 \leq \text{last}$, so that letting $y = x + 2m + 1$, we have $y + m \leq \text{last}$. Furthermore,

$y \in X \wedge X \cap]y, y + m] \neq \emptyset$, hence the pattern repeats. We define Φ_{near} as

$$\begin{aligned} \Phi_{\text{near}}(x, X) = & (x + 3m + 1 > \text{last}) \otimes \left((x \in X \wedge X \cap]x, x + m] \neq \emptyset) \oplus \right. \\ & \left. \bigoplus_{\substack{p \in Q, q_f \in F \\ m < d \leq 3m+1 \\ v = v_1 \dots v_d}} (X \cap]x, \text{last}] = \{x + p\} \wedge x + d - 1 = \text{last}) \otimes \Theta_{p, q_f}^v(x) \right). \end{aligned}$$

As FO-step formulas are closed by sum and product, the formula $\Phi(x, X)$ is an FO-step formula. Let us prove that $\llbracket \bigoplus_X \bigotimes_x \Phi(x, X) \rrbracket(u) = \llbracket \mathcal{A} \rrbracket(u)$. This equality has already been checked for words u of length at most m , so let us assume that $|u| > m$. In the following, we denote by $\text{Pat}(u)$ the set of all subsets $\{x_0 < t_0 < \dots < x_r < t_r\}$ with $t_0 = 2$, $x_\ell = (2m+1)\ell + 1$ and $1 \leq t_\ell - x_\ell \leq m$ for all $\ell \in \{0, \dots, r\}$, and $x_r + m \leq |u| < x_r + 3m + 1$. Let $q_\ell = t_\ell - x_\ell$. We have already explained that if $\llbracket \bigotimes_x \Phi(x, X) \rrbracket(u, I) \neq 0$, then we must have $I \in \text{Pat}(u)$. Then, for a word u of length greater than m , we get:

$$\begin{aligned} & \llbracket \bigoplus_X \bigotimes_x \Phi(x, X) \rrbracket(u) \\ &= \sum_{I \in \text{Pat}(u)} \prod_{1 \leq i \leq |u|} (\llbracket \Phi_{\text{far}} \rrbracket(u, i, I) + \llbracket \Phi_{\text{near}} \rrbracket(u, i, I)) \\ &= \sum_{I \in \text{Pat}(u)} \left(\prod_{1 \leq i \leq |u| - 3m - 1} \llbracket \Phi_{\text{far}} \rrbracket(u, i, I) \times \prod_{\max(1, |u| - 3m) \leq i \leq |u|} \llbracket \Phi_{\text{near}} \rrbracket(u, i, I) \right) \\ &= \sum_{I \in \text{Pat}(u)} \left(\prod_{0 \leq \ell \leq r-1} \llbracket \Theta_{q_\ell, q_{\ell+1}}^{u[x_\ell \dots x_{\ell+1} - 1]}(x) \rrbracket(u, x_\ell) \times \sum_{q_f \in F} (\llbracket \Theta_{q_r, q_f}^{u[x_r \dots |u|]}(x) \rrbracket(u, x_r)) \right). \end{aligned}$$

The last equality holds since $\llbracket \Phi_{\text{near}} \rrbracket(u, x_r, I) = \sum_{q_f \in F} (\llbracket \Theta_{q_r, q_f}^{u[x_r \dots |u|]}(x) \rrbracket(u, x_r))$ and since $\llbracket \Phi_{\text{near}} \rrbracket(u, i, I) = 1$ if $i \in \{\max(1, |u| - 3m), \dots, |u|\} \setminus \{x_r\}$. Finally, by (6),

$$\begin{aligned} \llbracket \bigoplus_X \bigotimes_x \Phi(x, X) \rrbracket(u) &= \sum_{\substack{q_0, \dots, q_r \in Q \\ q_0 = 1, q_f \in F}} \left(\prod_{0 \leq \ell \leq r-1} \mu(u[x_\ell \dots x_{\ell+1} - 1])_{q_\ell, q_{\ell+1}} \times \mu(u[x_r \dots |u|])_{q_r, q_f} \right) \\ &= \sum_{q_f \in F} \mu(u)_{q_0, q_f} = \llbracket \mathcal{A} \rrbracket(u). \end{aligned}$$

As in [Thomas 1982], we could use a more compact formula by encoding states in binary.

We finally prove 5 $\Rightarrow$ 7. Assume that Φ is of the form $\Phi(x, y) = \bigoplus_{i \in I} \varphi_i(x, y) \otimes s_i$. Again, we use the monadic second-order variable X to encode the *runs* of the weighted transitive closure operator. Let $u \in A^+$ be a word, $j \in \text{pos}(u)$ and $J \subseteq \text{pos}(u)$. If $j < \max(J)$ then we let $\text{next}(j, J) = \inf\{\ell \in J \mid j < \ell\}$. We define the MSO-step formula

$$\Phi'(x, X) \stackrel{\text{def}}{=} \bigoplus_{i \in I} s_i \otimes \exists y (x < y \wedge X \cap]x, y] = \{y\} \wedge \varphi_i(x, y))$$

and we can easily check that

$$\llbracket \Phi' \rrbracket(u, j, J) = \begin{cases} \llbracket \Phi \rrbracket(u, j, \text{next}(j, J)) & \text{if } j < \max(J) \\ 0 & \text{otherwise.} \end{cases}$$

The formula $\text{TC}_{x,y}^< \Phi(\text{first}, \text{last})$ is then equivalent to $\bigoplus_X \bigotimes_x (\Psi_{\text{small}} \oplus \Psi)$, with

$$\begin{aligned} \Psi &\stackrel{\text{def}}{=} (\text{first} \neq \text{last} \wedge \{\text{first}, \text{last}\} \subseteq X) \otimes ((x \in X \wedge x < \max(X)) \oplus \Phi'(x, X)) \\ \Psi_{\text{small}} &\stackrel{\text{def}}{=} (X = \emptyset \wedge \text{first} = \text{last}) \otimes ((x = \text{first}) \oplus \Phi(x, x)). \end{aligned}$$

Note that MSO-step formulas are closed under sum and products so the formula above is indeed in the required fragment. Let $u \in A^+$. In case, $|u| = 1$, we have $\llbracket \bigoplus_X \bigotimes_x (\Psi_{\text{small}} \oplus \Psi) \rrbracket(u) = \llbracket \mathcal{A} \rrbracket(u)$.

$\Psi\|)(u) = \llbracket \Phi \rrbracket(u, 1, 1) = \llbracket \text{TC}_{x,y}^< \Phi \rrbracket(u, 1, |u|)$. Otherwise, with the conditions on X in the formulas Ψ_{small} and Ψ we have

$$\llbracket \bigoplus_X \bigotimes_x (\Psi_{\text{small}} \oplus \Psi) \rrbracket(u) = \sum_{\{1, |u|\} \subseteq J \subseteq \{1, \dots, |u|\}} \llbracket \bigotimes_x \Psi \rrbracket(u, [X \mapsto J]).$$

For a choice of $J = \{j_0, j_1, \dots, j_k\}$ with $1 = j_0 < j_1 < \dots < j_k = |u|$, by definition of Ψ we have $\llbracket \bigotimes_x \Psi \rrbracket(u, [X \mapsto J]) = \prod_{0 \leq r < k} \llbracket \Phi \rrbracket(u, j_r, j_{r+1})$. Finally, we obtain

$$\begin{aligned} \llbracket \bigoplus_X \bigotimes_x (\Psi_{\text{small}} \oplus \Psi) \rrbracket(u) &= \sum_{k \geq 0} \sum_{1=j_0 < j_1 < \dots < j_k=|u|} \prod_{0 \leq r < k} \llbracket \Phi \rrbracket(u, j_r, j_{r+1}) \\ &= \llbracket \text{TC}_{x,y}^< \Phi \rrbracket(u, 1, |u|) \end{aligned}$$

where the last equality comes from the definition of the ordered weighted transitive closure operator. $\square$

5. PEBBLE WEIGHTED AUTOMATA

Example 3.2 shows that weighted automata lack closure properties to capture the logic $\text{wFOTC}^{\text{b}, <}(\text{FO})$. We introduce a notion of two-way pebble weighted automata making up for this gap. In the Boolean setting, two-way automata over strings have been first considered in [Rabin and Scott 1959], whereas pebble automata have been introduced in [Blum and Hewitt 1967].

5.1. General definitions

We now consider *pebble 2-way weighted automata* (P2WA). A P2WA has a read-only tape. At each step, it can move its head one position to the left or to the right (within the boundaries of the input tape), or drop a pebble at the current head position, or lift a pebble and resume the tape to its position. Applicable transitions and weights depend on the current letter and state, and on which pebbles are carried by the current position. Pebbles are numbered $p, \dots, 2, 1$, and are handled using a stack policy: if pebbles $p, \dots, \ell$ have already been dropped, the automaton can either lift pebble ℓ (if $\ell \leq p$), drop pebble $\ell - 1$ (if $\ell > 1$), or move.

As these automata can go in either direction, we add two fresh symbols $\triangleright$ and $\triangleleft$ to mark the beginning and the end of an input word. Let $\tilde{A} = A \uplus \{\triangleright, \triangleleft\}$. To compute the value of $u = u_1 \cdots u_n \in A^+$, a P2WA works on a tape holding $\triangleright u \triangleleft$. For convenience, we number the letters of $\triangleright u \triangleleft$ from 0, setting $u_0 = \triangleright$ and $u_{n+1} = \triangleleft$. We denote by $\widetilde{\text{pos}}(u)$ the set $\{0, 1, \dots, n, n+1\}$.

Definition 5.1. Let $p \geq 0$. A *pebble two-way weighted automaton* (P2WA) with p pebbles over $\mathbb{S}$ is a tuple $\mathcal{A} = (Q, A, I, F, \Delta, \text{weight})$ where Q is a finite set of states, A is a finite alphabet, $I \subseteq Q$ is the set of initial states, $F \subseteq Q$ is the set of final states, $\Delta \subseteq Q \times \tilde{A} \times 2^{\{1, \dots, p\}} \times D \times Q$ is the set of transitions, with $D = \{\rightarrow, \leftarrow, \text{drop}, \text{lift}\}$ and $\text{weight} : \Delta \rightarrow \mathbb{S}$ is the weight function.

A *configuration* of a P2WA $\mathcal{A}$ with p pebbles on a word $u \in A^+$ of length n is a triple $(q, \pi, i) \in Q \times \text{pos}(u)^{\leq p} \times \widetilde{\text{pos}}(u)$. In such a configuration, q denotes the current state of $\mathcal{A}$, while i is the current head position, and $\pi = \pi_p \cdots \pi_\ell$ with $1 \leq \ell \leq p+1$ encodes the locations of the $p+1-\ell$ currently dropped pebbles. More precisely, $\pi_m \in \{1, \dots, n\}$ is the position of pebble m , while pebbles $\ell-1, \dots, 1$ are currently not on the tape. Given such a word of pebbles $\pi = \pi_p \cdots \pi_\ell$, we denote by $\text{peb}(\pi, i)$ the set of pebbles currently dropped on position $i \in \widetilde{\text{pos}}(u)$, i.e., $\text{peb}(\pi, i) = \{j \mid p \geq j \geq \ell \wedge \pi_j = i\}$.

There is a *step* of weight s from configuration (q, π, i) to configuration (q', π', i') if there exists $d \in D$ such that $\delta \stackrel{\text{def}}{=} (q, u_i, \text{peb}(\pi, i), d, q') \in \Delta$ with $s = \text{weight}(\delta)$, and

$$\begin{cases} \pi' = \pi \text{ and } i' = i + 1 & \text{if } d = \rightarrow \\ \pi' = \pi \text{ and } i' = i - 1 & \text{if } d = \leftarrow \\ \pi' = \pi \cdot i \text{ and } i' = 0 & \text{if } d = \text{drop} \text{ and } i \in \text{pos}(u) \\ \pi' \cdot i' = \pi & \text{if } d = \text{lift} . \end{cases} \quad (7)$$

Note that a *drop* simply records the current position in u , with a fresh pebble (such a pebble should be available) and returns to the beginning of the tape, and that a *lift* returns on the last dropped pebble and pops it. This model corresponds to a weighted extension of the well-known *weak pebble automata* (as described, e.g., in [Bojańczyk et al. 2006]), as we can easily design *drop* transitions that do not reset the tape, and force *lift* transitions to occur only at the position of the topmost pebble. Notice that if it exists, the direction d is determined by the two configurations.

A *run* ρ of $\mathcal{A}$ on u is a sequence of configurations related by steps. We denote by $\text{weight}(\rho)$ the product of the weights of the steps of the run ρ (from left to right, but we will mainly work with commutative semirings in this section). A run is said to be *accepting* if it leads from a configuration $(q, \varepsilon, 0)$, with $q \in I$, to a configuration $(q', \varepsilon, n+1)$, with $q' \in F$ (at the end, no pebble is left on the tape). The run ρ is *simple* if whenever two configurations α and β appear in ρ , we have $\alpha \neq \beta$. The series $\llbracket \mathcal{A} \rrbracket \in \mathbb{S}\langle\langle A^+ \rangle\rangle$ is defined by $\llbracket \mathcal{A} \rrbracket(u) = \sum \text{weight}(\rho)$ where the sum ranges over all *simple* accepting runs ρ on u . Note that a P2WA with 0 pebble is in fact a 2-way weighted automaton. It follows from Proposition 5.9 below that this class of automata has the same expressive power as 1WA.

Example 5.2. Let us sketch a P2WA $\mathcal{A}$ with 1 pebble recognizing the series $u \mapsto 2^{|u|^2}$ over $\mathbb{N}$. The idea is that $\mathcal{A}$ drops its pebble successively on every position of the input word. Transitions for reallocating the pebble have weight 1. When a pebble is dropped, $\mathcal{A}$ scans the whole word from left to right where every transition has weight 2. As this scan happens $|u|$ times, we obtain $\llbracket \mathcal{A} \rrbracket(u) = 2^{|u|^2}$.

5.2. Pebble one-way weighted automata

The notion of simplicity is semantical. Our goal in this section is to find a syntactic restriction of P2WA enforcing simplicity. Mainly, it will consist in restricting to *1-way* runs, disabling left moves.

Definition 5.3. A *pebble one-way weighted automaton* (P1WA) over $\mathbb{S}$ is a P2WA $\mathcal{A} = (Q, A, I, F, \Delta, \text{weight})$ which satisfies the following conditions:

- (1) There are no left transitions: $(q, a, P, d, q') \in \Delta$ implies $d \neq \leftarrow$;
- (2) Lift transitions may not be followed by drop transitions: $(q, a, P, \text{lift}, q') \in \Delta$ and $(q', a', P', d', q'') \in \Delta$ imply $d' \neq \text{drop}$;

To justify (2), note that a lift transition followed by a drop transition would just result in resetting the current position to the beginning of the word, without dropping a new pebble.

The P2WA described in Example 5.2 is in fact a P1WA by definition.

As a fragment of P2WA, the semantics of a P1WA is well defined. However, the semantic restriction (considering only simple runs to define the weight of a word) is now useless, since every run of a P1WA is ensured to be simple, as shown in the next lemma.

LEMMA 5.4. *All runs of a P1WA are simple.*

PROOF. Consider a P1WA $\mathcal{A} = (Q, A, I, F, \Delta, \text{weight})$. Given a word u , we define a total order on the set of configurations of $\mathcal{A}$ over u , so that runs of $\mathcal{A}$ over u form an increasing sequence of configurations.

To do so, we consider the *measure* of a configuration (q, π, i) to be the word over the alphabet $\widetilde{\text{pos}}(u) \uplus \{\perp, \top\}$ defined by

$$\text{meas}(q, \pi, i) = \begin{cases} \pi \cdot i \cdot \perp & \text{if } \exists (q, a, P, \text{drop}, q') \in \Delta \\ \pi \cdot i \cdot \top & \text{otherwise.} \end{cases}$$

We order the set $\widetilde{\text{pos}}(u) \uplus \{\perp, \top\}$ by $\perp \prec 0 \prec 1 \prec \dots \prec |u| \prec |u| + 1 \prec \top$. Henceforth, finite words over this set are ordered with the associated lexicographic (total) order, denoted $\prec$.

We now show that for every run $\rho = (q_m, \pi_m, i_m)_{0 \leq m \leq h}$ of $\mathcal{A}$ over u , the sequence of measures of the configurations is increasing, i.e., $\text{meas}(q_m, \pi_m, i_m) \prec \text{meas}(q_{m+1}, \pi_{m+1}, i_{m+1})$ for all $0 \leq m < h$. This ensures that the run is simple. Let $0 \leq m < h$. The step from configuration $\gamma_m = (q_m, \pi_m, i_m)$ to configuration $\gamma_{m+1} = (q_{m+1}, \pi_{m+1}, i_{m+1})$ is enabled with a transition of the form $(q_m, u_{i_m}, \text{peb}(\pi_m, i_m), d, q_{m+1})$ with d defined as in (7) (specialized to the one-way case):

— If $d = \rightarrow$, then $\pi_{m+1} = \pi_m$ and $i_{m+1} = i_m + 1$. Hence, for some $\alpha, \beta \in \{\perp, \top\}$ we have

$$\text{meas}(\gamma_m) = \pi_m i_m \alpha \prec \pi_m (i_m + 1) \beta = \text{meas}(\gamma_{m+1}).$$

— If $d = \text{drop}$, then $\pi_{m+1} = \pi_m i_m$ and $i_{m+1} = 0$. Hence, for some $\alpha \in \{\perp, \top\}$ we have

$$\text{meas}(\gamma_m) = \pi_m i_m \perp \prec \pi_m i_m 0 \alpha = \text{meas}(\gamma_{m+1}).$$

— if $d = \text{lift}$, then $\pi_m = \pi_{m+1} i_{m+1}$. Hence, for some $\alpha \in \{\perp, \top\}$ we have

$$\text{meas}(\gamma_m) = \pi_{m+1} i_{m+1} i_m \alpha \prec \pi_{m+1} i_{m+1} \top = \text{meas}(\gamma_{m+1}).$$

Notice that the $\top$ in $\text{meas}(\gamma_{m+1})$ comes from the fact that lift transitions may not be followed by drop transitions, by definition of P1WA.

In all cases, we have $\text{meas}(\gamma_m) \prec \text{meas}(\gamma_{m+1})$, which concludes the proof. $\square$

Example 5.5 (Example 3.6 continued). The value computed to check that the counter always stays non-negative can also be calculated with the P1WA with 1 pebble of Figure 4: this automaton drops a pebble on each letter *Dec* and then computes the difference between the number of *Inc*'s and the number of *Dec*'s before the pebble. This is done as in Example 2.3. In the transitions of Figure 4 (the highlighted zone will be explained below), $*$ stands for any letter. We fix a word u , and denote by $i_1 < i_2 < \dots < i_m$ the positions of u labelled by *Dec*. Notice that each accepting run of the automaton drops the pebble on each position i_k (for every $1 \leq k \leq m$), and, after each such drop, must choose a position j_k to follow the transition from state 2 to state 3: moreover, we have $1 \leq j_k < i_k$ and $u_{j_k} \in \{\text{Inc}, \text{Dec}\}$. Denoting J_k the set of such positions j_k for every k , the set of accepting runs is therefore in bijection with the set $J = J_1 \times \dots \times J_m$. The weight of the run defined by the tuple $(j_k)_{1 \leq k \leq m}$ is then given by

$$\prod_{k=1}^m f(j_k)$$

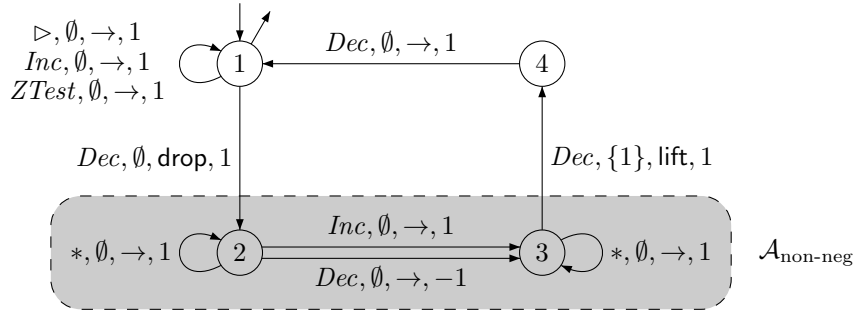


Fig. 4. Automaton checking that the counter is non-negative

where $f(j) = 1$ if $u_j = \text{Inc}$, -1 if $u_j = \text{Dec}$, and 0 otherwise. Hence, the semantics of the automaton over the word u is given by

$$\begin{aligned}
 \sum_{(j_k)_{1 \leq k \leq m} \in J} \prod_{k=1}^m f(j_k) &= \sum_{j_1 \in J_1} \sum_{j_2 \in J_2} \cdots \sum_{j_m \in J_m} \prod_{k=1}^m f(j_k) \\
 &= \prod_{k=1}^m \sum_{j_k \in J_k} f(j_k) \quad (\text{by distributivity of } \times \text{ over } +)^2 \\
 &= \prod_{k=1}^m (|u[1..i_k - 1]|_{\text{Inc}} - |u[1..i_k - 1]|_{\text{Dec}})
 \end{aligned}$$

Next, assuming that the counter always stays non-negative, the value checking that a zero-test is used only when the counter is indeed 0 can be computed with the P1WA with 2 pebbles of Figure 5 (highlighted zone to be explained below). Intuitively, it drops pebble 2 sequentially on every position performing action *ZTest* to compute the external product. When this pebble is dropped (over a position j), on every position k before the one holding the pebble (i.e., $k \leq j - 1$), it drops pebble 1 choosing non-deterministically either to compute

$$|u[1..j - 1]|_{\text{Inc}} - |u[1..j - 1]|_{\text{Dec}}$$

with the upper part, or $-k$ with the lower part. The proof of correctness is done similarly to the argument for the previous automaton, and strongly relies again on the distributivity of the multiplication over the addition.

5.3. One-way versus two-way

We proceed with the most difficult result of this paper: the equivalence between P2WA and P1WA when the semiring is commutative.

²Notice the transformation from the term on the left of the equality (a sum of products of weights) to this term (a product of sums of weights). The same transformation is actually behind the two alternative definitions of the semantics of a weighted automaton (see Section 2.2): whereas the definition based on runs is a sum of products, the one based on the morphism μ is a product of matrices. The idea yielding the second one from the first is to gather together runs with respect to common configurations: the weight of a word split into two factors uv is the product of the weights over u and v , each produced by summing the weights of all sub-runs over u and v , respectively. In the setting of pebble automata, we cannot simply split the word since the automaton may navigate and drop pebbles. However, if every accepting run goes through a given configuration exactly once, we may consider the sub-runs that start and end in this configuration, and group the weights similarly. In the present computation, we consider m such configurations, namely those where the pebble has just been dropped.

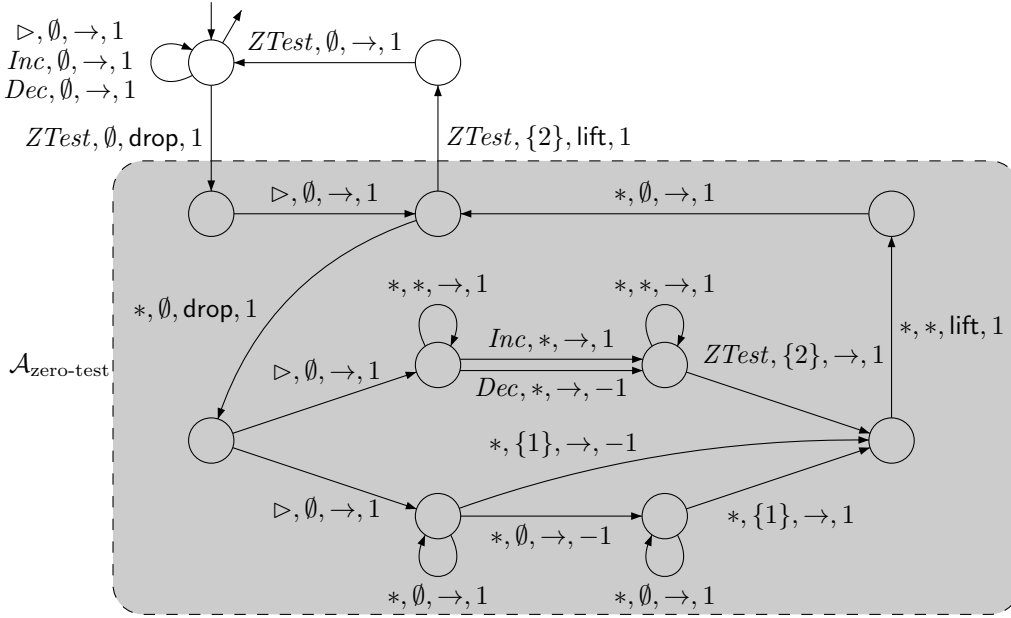


Fig. 5. Automaton checking the zero-tests

THEOREM 5.6. *Let $\mathbb{S}$ be a commutative semiring and $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$. Then, f is recognizable by a P2WA if and only if f is recognizable by a P1WA.*

One direction directly follows from the definitions of pebble weighted automata. The proof of the converse is by induction on the number of pebbles. We start with the base case of automata without pebbles. We simply write 2WA instead of “P2WA with 0 pebble”.

The next proposition is a generalization of the result, originally proved in [Rabin and Scott 1959], stating that two-way feature does not add expressive power to non-deterministic finite automata over finite strings. There are mainly two proof techniques for this result. The proof in [Shepherdson 1959] starts with a two-way automaton and constructs an equivalent one-way automaton by enriching the state with a relation table recording for every pair of states (q, q') , whether it is possible to reach q' from q with a loop on the current prefix. The key argument is that there is a finite number of such tables and that it can be computed simultaneously to the main run.

Observe that this method is not applicable in our weighted setting. Indeed, the relation table should now be enriched with the precomputed weights of the subruns, which cannot be recorded within a finite memory (as the weights may grow with the length of the prefix read so far). Instead, we will use the crossing-sequence method which is closer to the proof technique in [Rabin and Scott 1959], and fully explained in [Hopcroft and Ullman 1979]. For the sake of clarity, we provide a complete proof below in our weighted setting.

PROPOSITION 5.7. *Let $\mathbb{S}$ be a commutative semiring, $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$. If f is recognizable by a 2WA, then f is recognizable by a 1WA.*

PROOF. Let $\mathcal{A} = (R, A, I, F, \Delta, \text{weight})$ be a 2WA. We omit the (empty) set of pebbles in the transitions of the P2WA $\mathcal{A}$, i.e., $\Delta \subseteq R \times \tilde{A} \times D \times R$. We also assume that weight is fully defined on $R \times \tilde{A} \times D \times R$ by setting $\text{weight}(\delta) = 0$ if $\delta \notin \Delta$.

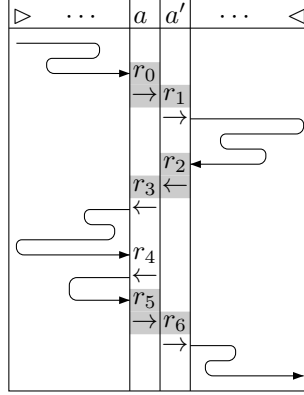


Fig. 6. Run of a 2WA and some crossing-sequences

We construct an equivalent 1WA (actually a P1WA without pebble³) $\mathcal{B} = (Q', A, I', F', \Delta', \text{weight}')$ such that $\llbracket \mathcal{B} \rrbracket = \llbracket \mathcal{A} \rrbracket$. Again, we omit the (empty) set of pebbles and the $(\rightarrow)$ direction in the transitions of the P1WA $\mathcal{B}$, i.e., $\Delta' \subseteq Q' \times \tilde{A} \times Q'$.

Intuitively, the idea is to record in a state of $\mathcal{B}$ the *crossing-sequence* of states of $\mathcal{A}$ which is observed in some accepting run ρ while scanning the current position of the input word, see Figure 6. Since the semantics of a 2WA is computed as the sum over *simple* accepting runs, a crossing-sequence consists of pairwise distinct states and is therefore bounded by the number of states of $\mathcal{A}$. Finally, the commutativity of the semiring allows us to compute the weight of an accepting run ρ along a corresponding run of $\mathcal{B}$.

In order to ease the matching of consecutive crossing-sequences of $\mathcal{A}$, we also record in a state of $\mathcal{B}$ the current input letter from $\tilde{A} = A \cup \{\triangleright, \triangleleft\}$ and the sequence of moves from $\{\leftarrow, \rightarrow\}$ performed by the 2WA. Therefore, we will define $Q' = Q'_{\triangleright} \cup Q'_A \cup Q'_{\triangleleft}$ as a *finite* subset of $T = \tilde{A}(R\{\leftarrow, \rightarrow\})^*(R \cup R\rightarrow)$. We say that a word in T is *R-simple* if it does not contain two occurrences of some state in R .

While scanning $\triangleright$ only right moves are possible hence we let $Q'_{\triangleright}$ be the (finite) set of *R-simple* words in $\triangleright(R\rightarrow)^+$. Similarly, while scanning $\triangleleft$, only left moves are possible until $\mathcal{A}$ finally halts on this position. Hence we let $Q'_{\triangleleft}$ be the (finite) set of *R-simple* words in $\triangleleft(R\leftarrow)^*R$. Finally, we let Q'_A be the (finite) set of *R-simple* words in $A(R\{\leftarrow, \rightarrow\})^*R\rightarrow$. For instance, Figure 6 exhibits two crossing-sequences in Q'_A which are $q = ar_0 \rightarrow r_3 \leftarrow r_4 \leftarrow r_5 \rightarrow$ and $q' = a'r_1 \rightarrow r_2 \leftarrow r_6 \rightarrow$.

We describe now the set $\Delta' \subseteq Q' \times \tilde{A} \times Q'$ of transitions of $\mathcal{B}$. Two states $a\tau$ and $a'\tau'$ in Q' can be *matched* in a transition $(a\tau, a, a'\tau') \in \Delta'$ of $\mathcal{B}$ if $a \neq \triangleleft$, $a' \neq \triangleright$ and τ has one more right moves than the number of left moves in τ' : $|\tau|_{\rightarrow} = 1 + |\tau'|_{\leftarrow}$.

The weight of a transition $(q, a, q') \in \Delta'$ is the product of the weights of the transitions of $\mathcal{A}$ *contained* in the pair (q, q') , i.e., right transitions from q to q' and left transitions from q' to q . On Figure 6, these transitions are on a gray background.

Formally, a right transition $(r, a, \rightarrow, r')$ of $\mathcal{A}$ is *contained* in (q, q') if r is the state just before the k th occurrence of $\rightarrow$ in q and r' the state just after the $(k-1)$ th occurrence⁴ of $\leftarrow$ in q' , for some $1 \leq k \leq |q|_{\rightarrow}$. Similarly, a left transition $(r', a', \leftarrow, r)$ of $\mathcal{A}$ is contained in (q, q') if r' the state just before the k th occurrence of $\leftarrow$ in q' and r is the state just after the k th occurrence of $\rightarrow$ in q , for some $1 \leq k \leq |q'|_{\leftarrow}$. For instance in Figure 6, the

³Note that, formally, a P1WA without pebble is not a 1WA as defined in Section 2.2. This is mainly due to the $\triangleright$ marker. But it should be clear that the two models are equivalent.

⁴In case $k = 1$, the state just after the 0th occurrence of $\leftarrow$ in q' is simply the first state of q' .

two right transitions from $\mathcal{A}$ contained in the depicted pair of states are $(r_0, a, \rightarrow, r_1)$ and $(r_5, a, \rightarrow, r_6)$, whereas the only left transition is $(r_2, a', \leftarrow, r_3)$.

We let $\text{Trans}(q, q')$ be the set of (left or right) transitions of $\mathcal{A}$ contained in (q, q') . We then define the weight of $\rightarrow$ moves of $\mathcal{B}$ induced by the matching states (q, q') as the product of the weights of the (left or right) transitions of $\mathcal{A}$ contained in (q, q') :

$$\text{weight}'(q, a, q') = \prod_{\delta \in \text{Trans}(q, q')} \text{weight}(\delta).$$

For instance, transition from $q = ar_0 \rightarrow r_3 \leftarrow r_4 \leftarrow r_5 \rightarrow$ to $q' = a'r_1 \rightarrow r_2 \leftarrow r_6 \rightarrow$ on Figure 6 has weight

$$\text{weight}'(q, a, q') = \text{weight}(r_0, a, \rightarrow, r_1) \times \text{weight}(r_2, a', \leftarrow, r_3) \times \text{weight}(r_5, a, \rightarrow, r_6).$$

Finally, the set I' of initial states of $\mathcal{B}$ is the set of R -simple words in $\triangleright I \rightarrow (R \rightarrow)^*$ and the set F' of final states of $\mathcal{B}$ is the set of R -simple words in $\triangleleft (R \leftarrow)^* F$.

Lemma 5.8 below proves that there is a weight preserving bijection between the simple accepting runs of $\mathcal{A}$ and the accepting runs of $\mathcal{B}$. We conclude that $\llbracket \mathcal{B} \rrbracket = \llbracket \mathcal{A} \rrbracket$ since for all $u \in A^+$, $\llbracket \mathcal{A} \rrbracket(u)$ is the sum of the weights of the simple accepting runs of $\mathcal{A}$ over u , whereas $\llbracket \mathcal{B} \rrbracket(u)$ is the sum of the weights of the accepting runs of $\mathcal{B}$ over u . $\square$

LEMMA 5.8. *For every word $u \in A^+$, there is a weight preserving bijection between the simple accepting runs of $\mathcal{A}$ over u and the accepting runs of $\mathcal{B}$ over u .*

PROOF. Fix a word $u = u_1 \cdots u_n \in A^+$ of length $n \geq 1$. We first show how to map a simple accepting run $\rho = \rho_0 \rho_1 \cdots \rho_m$ of $\mathcal{A}$ over u (where each ρ_j is a configuration) to an accepting run of $\mathcal{B}$ over u .

For $i \in \{0, \dots, n+1\}$, we construct the crossing-sequence $q_i = u_i \tau_i \in Q'$ of $\mathcal{B}$ associated with position i in $\triangleright u \triangleleft$, as illustrated in Figure 6. Recall that $u_0 = \triangleright$ and $u_{n+1} = \triangleleft$. Formally, we define τ_i as the “projection” of the configurations visiting position i , where configuration (r, i) is “projected” to $r \rightarrow$ (respectively, $r \leftarrow$ or r) if it is followed by some $(r', i+1)$ (respectively, $(r', i-1)$ or is the last configuration). Note that the word τ_i is R -simple. In particular, we have $q_0 \in I'$, $q_{n+1} \in F'$, and $q_i \in Q'_A$ for every $1 \leq i \leq n$.

Observe also that $|\tau_i|_{\rightarrow} = 1 + |\tau_{i+1}|_{\leftarrow}$ for every $0 \leq i \leq n$. Indeed, ρ must reach position $i+1$ for a first time, and then, each time the run ρ goes to the left of position $i+1$, it will in the future come back to $i+1$ from i . Therefore, $(q_i, u_i, q_{i+1}) \in \Delta'$ for all $0 \leq i \leq n$. We deduce that $f(\rho) = (q_0, 0)(q_1, 1) \cdots (q_{n+1}, n+1)$ is an accepting run of $\mathcal{B}$.

We now show that f preserves the weights, i.e., $\text{weight}(\rho) = \text{weight}'(f(\rho))$ for all simple accepting run ρ of $\mathcal{A}$ over u . Indeed, $\text{weight}(\rho)$ is the product of the weights of (left or right) transitions appearing in ρ . With $f(\rho) = (q_0, 0)(q_1, 1) \cdots (q_{n+1}, n+1)$, every right transition of ρ starting from position i ($0 \leq i \leq n$) is contained in the pair (q_i, q_{i+1}) , so that its weight is computed in the transition from i to $i+1$ in $f(\rho)$, whereas every left transition of ρ starting from position i ($1 \leq i \leq n+1$) is contained in the pair (q_{i-1}, q_i) , so that its weight is computed in the transition from $i-1$ to i in $f(\rho)$. From the definition of weight' and by commutativity of $\mathbb{S}$, we obtain $\text{weight}(\rho) = \text{weight}'(f(\rho))$.

We finally prove that f is a bijection by constructing its inverse function. Let $(q_0, 0)(q_1, 1) \cdots (q_{n+1}, n+1)$ be an accepting run of $\mathcal{B}$ over u . Write $q_i = u_i \tau_i$ for $0 \leq i \leq n+1$.

We recover a simple accepting run ρ of $\mathcal{A}$ over u as the output of $\text{Run}(0, \tau_0, \tau_1, \dots, \tau_{n+1})$ where the recursive function Run is defined by:

Function $\text{Run}(i, \tau_0, \tau_1, \dots, \tau_{n+1})$

```

match  $\tau_i$  with
|  $r \rightarrow \tau'_i$  :
|   output  $(r, i)$ ;
|   Run  $(i + 1, \tau_0, \dots, \tau'_i, \dots, \tau_{n+1})$ ;
|  $r \leftarrow \tau'_i$  :
|   output  $(r, i)$ ;
|   Run  $(i - 1, \tau_0, \dots, \tau'_i, \dots, \tau_{n+1})$ ;
|  $r : (* \text{ necessarily } i = n + 1 *)$ 
|   output  $(r, n + 1)$ ;

```

We can show by induction that $f(\text{Run}(0, \tau_0, \tau_1, \dots, \tau_{n+1})) = (q_0, 0)(q_1, 1) \cdots (q_{n+1}, n + 1)$, and that for every simple accepting run ρ of $\mathcal{A}$ with $f(\rho) = (u_0\tau_0, 0) \cdots (u_{n+1}\tau_{n+1}, n + 1)$, we have $\text{Run}(0, \tau_0, \dots, \tau_{n+1}) = \rho$. Therefore, f is a bijection. $\square$

The proof of Proposition 5.7 may be easily adapted to various extensions of 2WA. For instance, instead of restricting to *simple* runs when computing the semantics of a 2WA $\mathcal{A}$, we may allow k -simple runs (for some fixed k) in which no configuration is visited more than k times. Even when $k = 2$, we do not know an easy way to construct a 2WA $\mathcal{A}'$ whose semantics over 1-simple runs coincide with the semantics of $\mathcal{A}$ over 2-simple runs. But the proof of Proposition 5.7 allows to cope with this extension simply by allowing k -simple words when defining the set Q' of states of $\mathcal{B}$. Hence, we can construct a 1WA $\mathcal{B}$ whose semantics coincides with the semantics of $\mathcal{A}$ over k -simple runs, showing that this extension does not add expressive power.

Instead of allowing more runs, one may also wish to restrict the semantics of $\mathcal{A}$ to fewer runs, e.g., those visiting the initial position $\triangleright$ no more than 3 times. To do so, the natural approach is to add some information to the states of $\mathcal{A}$, e.g., a counter recording how many times (limited to 3) the initial position has been visited. But simple runs of the resulting 2WA $\mathcal{A}'$ do not necessarily correspond to simple runs of $\mathcal{A}$. Indeed the configurations $((p, 1), 8)$ and $((p, 2), 8)$ of $\mathcal{A}'$ are distinct but if we project away the counter we get the same configuration $(p, 8)$ of $\mathcal{A}$. To obtain the desired semantics, one may introduce an equivalence relation $\sim$ on the states of $\mathcal{A}'$ (having the same state from $\mathcal{A}$) and restrict the semantics of $\mathcal{A}'$ to $\sim$ -simple runs, i.e., runs which do not visit two $\sim$ -equivalent configurations. Again, the proof of Proposition 5.7 allows to cope with the $\sim$ -simple semantics of a 2WA, by restricting the set Q' of states of $\mathcal{B}$ to $\sim$ -simple words.

Finally, we introduce another extension which will be useful in the proof of Proposition 5.9 below. We generalize the acceptance condition of a 2WA $\mathcal{A} = (R, A, I, F, \Delta, \text{weight})$ by letting I, F be finite subsets of R^+ . A run is declared accepting if the sequence of states visiting the initial position $\triangleright$ is in I and similarly the sequence of states visiting the final position $\triangleleft$ is in F . Notice that this is indeed a generalization as every 2WA $\mathcal{A} = (R, A, I, F, \Delta, \text{weight})$ can be encoded into this new formalism by replacing I (respectively, F) with the (finite) set of all simple sequences of states starting with a state of I (respectively, ending with a state of F). Again, the proof of Proposition 5.7 allows to cope with this extension easily by changing I' to the sequences in $Q'_{\triangleright}$ whose projections on R are in $I \subseteq R^+$, and similarly for F' .

We extend now the translation of Proposition 5.7 to weighted automata with pebbles. The general idea is an induction on the number of pebbles, but it has to be done with great care so that all and only the *simple* runs of the P2WA are taken into account.

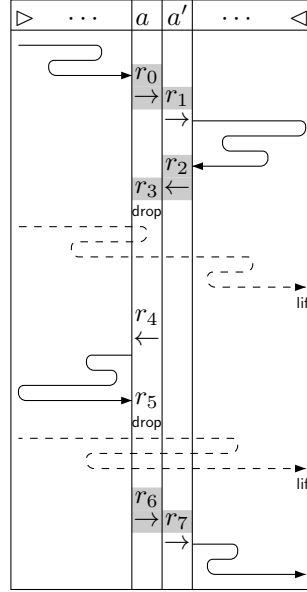


Fig. 7. Run of a 2WA and some crossing-sequences

PROPOSITION 5.9. *Let $\mathbb{S}$ be a commutative semiring, $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$ and $p \geq 0$. If f is recognizable by a P2WA with p pebbles, then f is recognizable by a P1WA with p pebbles.*

PROOF. We start with a P2WA $\mathcal{A} = (R, A, I, F, \Delta, \text{weight})$ with p pebbles, and a generalized acceptance condition: I and F are finite subsets of R^+ . A run ρ of $\mathcal{A}$ is accepting if the sequence of states visiting the initial position $\triangleright$ while no pebbles are dropped is in I , and the sequence of states visiting the final position $\triangleleft$ while no pebbles are dropped is in F . This generalization will be useful in the inductive step of the construction. We assume without loss of generality that lift transitions of $\mathcal{A}$ only occur at the end marker $\triangleleft$.

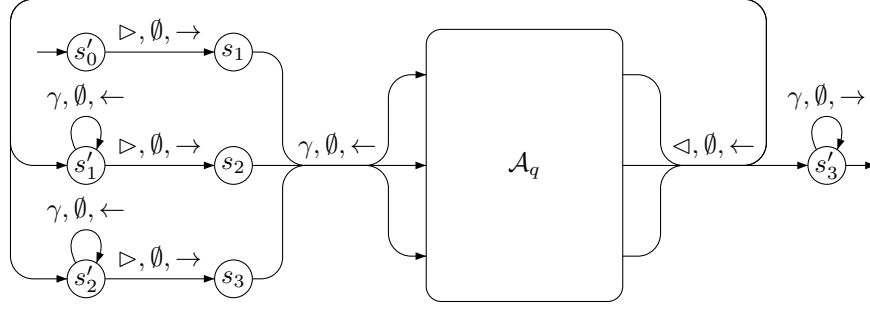
If $p = 0$, Proposition 5.7 permits to conclude. We now explain the construction for $p > 0$. A crossing-sequence may now also include drop moves, see Figure 7. The parts of a run having at least one pebble dropped (dashed in Figure 7) will be deferred to the inductive step. Hence, a crossing-sequence consists only of the states visiting some position when no pebbles are dropped. If the run is simple, then this sequence of states is also simple.

As in the proof of Proposition 5.7, we also include the current letter and the sequence of moves in a crossing-sequence. Hence, we let $Q'_{\triangleright}$ be the (finite) set of R -simple words in $\triangleright(R \rightarrow)^+$, $Q'_{\triangleleft}$ be the (finite) set of R -simple words in $\triangleleft(R \leftarrow)^* R$, and Q'_A be the (finite) set of R -simple words in $A(R\{\leftarrow, \rightarrow, \text{drop}\})^* R \rightarrow$. For instance, Figure 7 exhibits the crossing-sequence $q = ar_0 \rightarrow r_3 \text{drop } r_4 \leftarrow r_5 \text{drop } r_6 \rightarrow \in Q'_A$ in which the states r_0, r_3, r_4, r_5, r_6 must be pairwise distinct.

For each state $q = a\tau \in Q'_A$, we construct a P2WA $\mathcal{A}_q = (R_q, \Gamma, I_q, F_q, \Delta_q, \text{weight}_q)$ with $(p - 1)$ pebbles which computes the sum of the weights of the tuples of runs filling the drop-lift gaps in the crossing-sequence q , i.e., the dashed lines in Figure 7.

The input alphabet of $\mathcal{A}_q$ is $\Gamma = A \times \{0, 1\}$ where the second component encodes the position of the dropped pebble. Here, we are only interested in words with a unique position carrying a 1 in this second component of $A \times \{0, 1\}$: if i is this unique position, we denote as (u, i) such a word. Recall that $\tilde{\Gamma} = \Gamma \cup \{\triangleright, \triangleleft\}$.

From a crossing-sequence $q = a\tau \in Q'_A$, we first extract the sequence $\sigma(\tau)$ of pairs of states surrounding drop moves. For instance, $\sigma(r_0 \rightarrow r_3 \text{drop } r_4 \leftarrow r_5 \text{drop } r_6 \rightarrow) =$

Fig. 8. The P2WA $\mathcal{A}_q$ filling the drop-lift gaps of the crossing-sequence q

$(r_3, r_4)(r_5, r_6)$ and $\sigma(r_1 \text{drop } r_2 \text{drop } r_3 \rightarrow) = (r_1, r_2)(r_2, r_3)$. We assume below that $\sigma(\tau) = (r_1, r'_1)(r_2, r'_2) \cdots (r_N, r'_N)$ with $N > 0$. Note that $N \leq |R|$ since $r_1, r_2, \dots, r_N$ must be pairwise distinct.

We let $R_q = R \uplus \{s'_0, s_1, s'_1, \dots, s_N, s'_N\}$ be the set of states of $\mathcal{A}_q$. The automaton $\mathcal{A}_q$ should fill the drop-lift gaps between (r_j, r'_j) for all $1 \leq j \leq N$. To do so, it simulates $\mathcal{A}$ interpreting pebble p as the $\{0, 1\}$ component of Γ (recall that pebble p is the first which is dropped by $\mathcal{A}$). Hence, for $(r, \gamma, P, d, r') \in \Delta_q$ with $r, r' \in R$, $\gamma \in \bar{\Gamma}$, $P \subseteq \{1, \dots, p-1\}$ and $d \in D$ we let

$$\begin{aligned} \text{weight}_q(r, (c, 1), P, d, r') &= \text{weight}(r, c, P \cup \{p\}, d, r') && \text{if } \gamma = (c, 1) \\ \text{weight}_q(r, (c, 0), P, d, r') &= \text{weight}(r, c, P, d, r') && \text{if } \gamma = (c, 0) \\ \text{weight}_q(r, \gamma, P, d, r') &= \text{weight}(r, \gamma, P, d, r') && \text{if } \gamma \in \{\triangleright, \triangleleft\} \text{ (and } P = \emptyset\text{)}. \end{aligned}$$

The additional states in $\{s'_0, s_1, s'_1, \dots, s_N, s'_N\}$ and transitions will make sure that all gaps (r_j, r'_j) are filled, see Figure 8. States s'_{j-1} are used to reset the head to the initial position $\triangleright$ between two simulations, and we move from s'_{j-1} to s_j at the beginning of the word:

$$\begin{aligned} \text{weight}_q(s'_{j-1}, \gamma, \emptyset, \leftarrow, s'_{j-1}) &= 1 && \text{for } 1 < j \leq N \text{ and } \gamma \in \bar{\Gamma} \\ \text{weight}_q(s'_{j-1}, \triangleright, \emptyset, \rightarrow, s_j) &= 1 && \text{for } 1 \leq j \leq N. \end{aligned}$$

A drop transition of $\mathcal{A}$ from state r_j reading letter a (recall $q = a\tau$) is simulated in $\mathcal{A}_q$ by a $\leftarrow$ transition from s_j reading the first letter γ of the word (note that when $\mathcal{A}_q$ is in state s_j it must be on the first position of the word due to the $\rightarrow$ transition from s'_{j-1} to s_j). A lift transition of $\mathcal{A}$ reaching r'_j is simulated in $\mathcal{A}_q$ by a $\leftarrow$ transition to s'_j :

$$\begin{aligned} \text{weight}_q(s_j, \gamma, \emptyset, \leftarrow, r) &= \text{weight}(r_j, a, \emptyset, \text{drop}, r) && \text{for } 1 \leq j \leq N, r \in R \text{ and } \gamma \in \bar{\Gamma} \\ \text{weight}_q(r', \triangleleft, \emptyset, \leftarrow, s'_j) &= \text{weight}(r', \triangleleft, \emptyset, \text{lift}, r'_j) && \text{for } 1 \leq j \leq N \text{ and } r' \in R. \end{aligned}$$

Finally, when $\mathcal{A}_q$ reaches s'_N , it terminates with a $\rightarrow$ move:

$$\text{weight}_q(s'_N, \gamma, \emptyset, \rightarrow, s'_N) = 1 \quad \text{for } \gamma \in \bar{\Gamma}.$$

To define the acceptance condition of $\mathcal{A}_q$, we let I_q be the (finite) set of *simple* sequences in $s'_0 R^* s'_1 R^* \cdots s'_{N-1} R^*$ and F_q be the (finite) set of *simple* sequences in $R^* s'_N$. Considering this acceptance condition, an accepting run $\hat{\rho}$ of $\mathcal{A}_q$ — over a word of Γ^+ with a unique position carrying a 1 in its second component — can be split into a sequence of runs $\hat{\rho} = \hat{\rho}_0 \hat{\rho}_1 \hat{\rho}'_1 \cdots \hat{\rho}_N \hat{\rho}'_N$ where for each $1 \leq j \leq N$

- the run $\hat{\rho}_j$ fills the drop-lift gap (r_j, r'_j) . It goes from s_j to s'_j and simulates a run ρ_j of $\mathcal{A}$ which starts with a drop transition $(r_j, a, \emptyset, \text{drop}, r)$ and ends with a lift transition

- $(r', \triangleleft, \emptyset, \text{lift}, r'_j)$ without lifting pebble p in-between. Moreover, $\text{weight}_q(\hat{\rho}_j) = \text{weight}(\rho_j)$ for all $1 \leq j \leq N$.
- the run ρ'_0 consists of a single transition from s'_0 to s_1 , the run ρ'_j for $0 < j < N$ loops on state s'_j to reset the head to the initial position $\triangleright$ and then moves to state s_j , and the run ρ'_N consists of a single transition looping on s'_N to reach the final position $\triangleleft$. Moreover, $\text{weight}_q(\rho'_j) = 1$ for all $0 \leq j \leq N$.

Note that, the run $\hat{\rho}$ is simple if and only if the sequence of runs $\rho_1, \dots, \rho_N$ is *globally* simple, i.e., a configuration may not occur twice in some ρ_j and it may not occur both in ρ_i and ρ_j with $i \neq j$. This is important since the runs $\rho_1, \dots, \rho_N$ fill the **drop-lift** gaps of the crossing-sequence $a\tau$ and are therefore part of a single run of $\mathcal{A}$. This explains why we have a single copy of $\mathcal{A}$ in $\mathcal{A}_q$. If instead we used N copies of $\mathcal{A}$ inside $\mathcal{A}_q$, one for each **drop-lift** gap (r_j, r'_j) then the simplicity of $\hat{\rho}$ would not imply the *global* simplicity⁵ of $\rho_1, \dots, \rho_N$.

Clearly, all accepting runs of $\mathcal{A}_q$ start from state s'_0 and end in state s'_N . But this basic initial and final condition is not sufficient to ensure that the **drop-lift** gaps are filled correctly. For instance, $\mathcal{A}_q$ could go directly from s'_0 to s'_N without ever visiting $s'_1, \dots, s'_{N-1}$. It could also visit all the new states but not in the intended order, which would not correspond to a sequence of runs of $\mathcal{A}$ filling the gaps $(r_1, r'_1) \cdots (r_N, r'_N)$. This is why we use the generalized acceptance condition for I_q .

Towards a uniform construction below, we also define an automaton $\mathcal{A}_q$ when the crossing-sequence q has no drop ($N = 0$). In this case, $\mathcal{A}_q$ has a single state s'_0 with a loop moving right with weight 1 in order to simply scan the word from begin to end.

By induction, for each $q \in Q'_A$ we can construct a P1WA $\mathcal{B}_q$ with $(p-1)$ pebbles which is equivalent to $\mathcal{A}_q$, i.e., such that for each word $u \in \Gamma^+$ there is a weight preserving bijection between the simple accepting runs of $\mathcal{A}_q$ over u and the accepting runs of $\mathcal{B}_q$ over u . We let $\mathcal{B}_q = (Q'_q, \Gamma, I'_q, F'_q, \Delta'_q, \text{weight}'_q)$ and we assume without loss of generality that $I'_q = \{\text{init}_q\}$ and $F'_q = \{\text{final}_q\}$ are singletons. Note that, if the crossing-sequence q has no drop then we may choose $\mathcal{B}_q = \mathcal{A}_q$.

We define now the P1WA $\mathcal{B} = (Q', A, I', F', \Delta', \text{weight}')$ with p pebbles associated with $\mathcal{A}$. Its set of states is

$$Q' = Q'_\triangleright \uplus Q'_\triangleleft \uplus Q'_A \uplus \overline{Q'_A} \uplus \biguplus_{q \in Q'_A} Q'_q.$$

The set I' of initial states of $\mathcal{B}$ is the set of crossing-sequences in $Q'_\triangleright$ whose projections on R are in $I \subseteq R^+$, and similarly, the set F' of final states of $\mathcal{B}$ is the set of crossing-sequences in $Q'_\triangleleft$ whose projections on R are in $F \subseteq R^+$.

The automaton $\mathcal{B}$ contains a copy of each $\mathcal{B}_q$ in order to simulate the parts of a run when the first pebble is dropped (dashed lines in Figure 7). In this copy, the second component of the alphabet $\Gamma = A \times \{0, 1\}$ of $\mathcal{B}_q$ is re-interpreted as the encoding of the first pebble p : for $s, s' \in Q'_q$, $c \in A$ and $P \subseteq \{1, \dots, p-1\}$ we set

$$\begin{aligned} \text{weight}'(\text{init}_q, \triangleright, \emptyset, \rightarrow, s') &= \text{weight}'_q(\text{init}_q, \triangleright, \emptyset, \rightarrow, s') \\ \text{weight}'(s, c, P, \rightarrow, s') &= \text{weight}'_q(s, (c, 0), P, \rightarrow, s') \\ \text{weight}'(s, c, P \cup \{p\}, \rightarrow, s') &= \text{weight}'_q(s, (c, 1), P, \rightarrow, s'). \end{aligned}$$

From each state $q = a\tau \in Q'_A$, we first call (the copy of) the automaton $\mathcal{B}_q$ dropping the first pebble and when the computation of $\mathcal{B}_q$ is done we lift the pebble returning to the copy

⁵Alternatively, we could define an equivalence relation $\sim$ on states of $\mathcal{A}_q$ and use a $\sim$ -simple semantics of runs, as originally done in [Bollig et al. 2010].

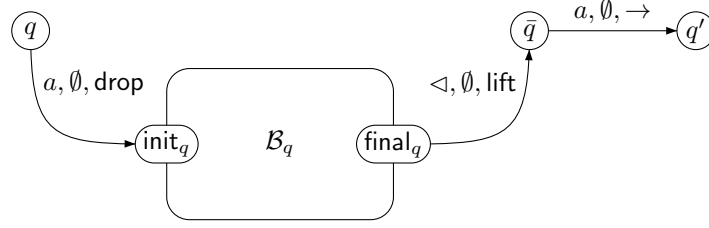


Fig. 9. Call to the P1WA $\mathcal{B}_q$ filling the drop-lift gaps of the crossing-sequence q

$\bar{q} \in \overline{Q'_A}$ of $q \in Q'_A$, see Figure 9. Then, we perform a $\rightarrow$ move to the next crossing-sequence $q' \in Q'_A \cup Q'_{\triangleleft}$. The **drop** and **lift** moves are with weight 1:

$$\text{weight}'(q, a, \emptyset, \text{drop}, \text{init}_q) = \text{weight}'(\text{final}_q, \triangleleft, \emptyset, \text{lift}, \bar{q}) = 1.$$

As in the proof of Proposition 5.7, two crossing-sequences $q = a\tau \in Q'_{\triangleright} \cup Q'_A$ and $q' = a'\tau' \in Q'_A \cup Q'_{\triangleleft}$ match if $|\tau|_{\rightarrow} = 1 + |\tau'|_{\leftarrow}$. We assume $\text{Trans}(q, q')$ to be defined as in Proposition 5.7, so that we may define the weight of $\rightarrow$ moves of $\mathcal{B}$ induced by the matching crossing-sequences (q, q') as the product of the weights of the (left or right) transitions of $\mathcal{A}$ contained in (q, q') :

$$\begin{aligned} \text{weight}'(q, \triangleright, \emptyset, \rightarrow, q') &= \prod_{\delta \in \text{Trans}(q, q')} \text{weight}(\delta) && \text{if } q \in Q'_{\triangleright} \\ \text{weight}'(\bar{q}, a, \emptyset, \rightarrow, q') &= \prod_{\delta \in \text{Trans}(q, q')} \text{weight}(\delta) && \text{if } q = a\tau \in Q'_A. \end{aligned}$$

Lemma 5.10 below proves that there is a weight preserving bijection between the simple accepting runs of $\mathcal{A}$ and the accepting runs of $\mathcal{B}$. We conclude that $\llbracket \mathcal{B} \rrbracket = \llbracket \mathcal{A} \rrbracket$ since for all $u \in A^+$, $\llbracket \mathcal{A} \rrbracket(u)$ is the sum of the weights of the simple accepting runs of $\mathcal{A}$ over u , whereas $\llbracket \mathcal{B} \rrbracket(u)$ is the sum of the weights of the accepting runs of $\mathcal{B}$ over u . $\square$

LEMMA 5.10. *For every word $u \in A^+$, there is a weight preserving bijection between the simple accepting runs of $\mathcal{A}$ over u and the accepting runs of $\mathcal{B}$ over u .*

PROOF. Fix a word $u = u_1 \cdots u_n \in A^+$ of length $n \geq 1$. We will decompose our bijection into several ones, all preserving the weights in an adequate manner.

To extend the proof of Lemma 5.8, we first associate with a simple accepting run ρ of $\mathcal{A}$ over u a tuple $f(\rho) = (q_0, q_1, g_1, \dots, q_i, g_i, \dots, q_n, g_n, q_{n+1})$ where $q_i = u_i \tau_i$ is the crossing-sequence at position $0 \leq i \leq n+1$ and for each position $1 \leq i \leq n$, the tuple $g_i = (\rho_1^i, \dots, \rho_{N_i}^i)$ consists of the sequence of subruns of ρ with pebble p dropped on position i , together with the surrounding drop/lift transitions.

Formally, for $0 \leq i \leq n+1$, to define the crossing sequence $q_i = u_i \tau_i$, we let τ_i be the “projection” of the configurations visiting position i with an empty stack of pebbles, where configuration (r, ε, i) is “projected” to $r \rightarrow$ (respectively, $r \leftarrow$, $r \text{ drop}$ or r) if it is followed by some $(r', \varepsilon, i+1)$ (respectively, $(r', \varepsilon, i-1)$, $(r', i, 0)$ or is the last configuration).

Moreover, for $1 \leq i \leq n$ with $N_i = |q_i|_{\text{drop}}$, we extract from ρ the tuple $g_i = (\rho_1^i, \dots, \rho_{N_i}^i)$ of subruns of the form $(r_j, \varepsilon, i)(r_{j+1}, \pi_{j+1}, 0) \cdots (r_{k-1}, \pi_{k-1}, n+1)(r_k, \varepsilon, i)$ where $\pi_\ell = i \cdot \pi'_\ell$ for all $j < \ell < k$, i.e., where the first pebble is dropped on position i .

We first show that f is a bijection from simple accepting runs of $\mathcal{A}$ over u to the set, denoted Seq_u , of sequences $(q_0, q_1, g_1, \dots, q_i, g_i, \dots, q_n, g_n, q_{n+1})$ verifying:

- $q_0 \in I'$, $q_{n+1} \in F'$ and $q_i \in Q'_A$ for all $1 \leq i \leq n$;
- for all $0 \leq i \leq n$, the pair (q_i, q_{i+1}) matches, i.e., $|q_i|_{\rightarrow} = 1 + |q_{i+1}|_{\leftarrow}$;

— for all $1 \leq i \leq n$, denoting $N_i = |q_i|_{\text{drop}}$, g_i is a tuple $(\rho_1^i, \dots, \rho_{N_i}^i)$ of subruns of $\mathcal{A}$, globally simple, each starting with the drop of pebble p on position i , ending with the first time this pebble is lifted, and that *matches* with $\sigma(q_i)$ (i.e., the j th pair in $\sigma(q_i)$ contains exactly the pair of first and last states of the run ρ_j^i).

It is not difficult to check that $f(\rho) \in \text{Seq}_u$ whenever ρ is a simple accepting run of $\mathcal{A}$ over u . Defining the weight of a sequence $(q_0, q_1, g_1, \dots, q_n, g_n, q_{n+1}) \in \text{Seq}_u$ as

$$\text{weight}'(q_0, \triangleright, \emptyset, \rightarrow, q_1) \times \prod_{i=1}^n (\text{weight}(g_i) \times \text{weight}'(\bar{q}_i, u_i, \emptyset, \rightarrow, q_{i+1}))$$

where $\text{weight}(g_i) = \prod_{j=1}^{N_i} \text{weight}(\rho_j^i)$ with $g_i = (\rho_1^i, \dots, \rho_{N_i}^i)$, we can easily show that f preserves the weights.

We finally prove that f is a bijection by constructing its inverse function. Let $(q_0, q_1, g_1, \dots, q_n, g_n, q_{n+1}) \in \text{Seq}_u$. Write $q_i = u_i \tau_i$ for $0 \leq i \leq n+1$. We recover a simple accepting run ρ of $\mathcal{A}$ over u as the output of $\text{Run}'(0, \tau_0, \tau_1, g_1, \dots, \tau_n, g_n, \tau_{n+1})$ where the recursive function Run' is defined by:

Function $\text{Run}'(i, \tau_0, \tau_1, g_1, \dots, \tau_n, g_n, \tau_{n+1})$

Data: A position i of u and a tuple $(\tau_0, \tau_1, g_1, \dots, \tau_n, g_n, \tau_{n+1})$
Result: A run ρ of $\mathcal{A}$

match τ_i **with**

- $r \rightarrow \tau_i' :$
 - output** $(r, \varepsilon, i);$
 - $\text{Run}'(i+1, \tau_0, \dots, \tau_i', g_i, \dots, \tau_{n+1});$
- $r \leftarrow \tau_i' :$
 - output** $(r, \varepsilon, i);$
 - $\text{Run}'(i-1, \tau_0, \dots, \tau_i', g_i, \dots, \tau_{n+1});$
- $r \text{drop } \tau_i' :$
 - match** g_i **with**
 - $(\rho', g_i') :$
 - output** $\rho';$
 - $\text{Run}'(i, \tau_0, \dots, \tau_i', g_i', \dots, \tau_{n+1});$
- $r : (* \text{ necessarily } i = n+1 *)$
 - output** $(r, \varepsilon, n+1);$

We then consider each tuple g_i of runs of $\mathcal{A}$ with pebble p dropped. It has already been explained in the previous proof how to construct a weight preserving bijection between a tuple $g_i = (\rho_1^i, \dots, \rho_{N_i}^i)$, which matches with a sequence of pairs $\sigma(q_i)$, and an accepting run $\hat{\rho}_i = \rho_0' \hat{\rho}_1^i \rho_1' \dots \hat{\rho}_{N_i}^i \rho_{N_i}'$ of $\mathcal{A}_{q_i}$ over (u, i) so that g_i is globally simple if and only if $\hat{\rho}_i$ is simple: it consists of filling some gaps (with the runs ρ_j^i), and replacing alphabet A by $\Gamma = A \times \{0, 1\}$ (going from runs ρ_j^i to $\hat{\rho}_j^i$).

Using the induction hypothesis, we know that there is a weight preserving bijection between simple accepting runs $\hat{\rho}_i$ of $\mathcal{A}_{q_i}$ over (u, i) and accepting runs $\bar{\rho}_i$ of $\mathcal{B}_{q_i}$ over (u, i) .

It remains to change back the alphabet to A , to build a weight preserving bijection between accepting runs $\bar{\rho}_i$ of $\mathcal{B}_{q_i}$ over (u, i) to runs $\bar{g}_i$ of $\mathcal{B}$ over u starting in state init_{q_i} in position 0 and finishing in state final_{q_i} in position $n+1$. Finally, coming back to the sequences of Seq_u , these intermediary bijections permit to build a weight preserving bijection from sequences $(q_0, q_1, g_1, \dots, q_i, g_i, \dots, q_n, g_n, q_{n+1}) \in \text{Seq}_u$ to accepting runs

$(q_0, \varepsilon, 0)(q_1, \varepsilon, 1)\bar{g}_1(\bar{q}_1, \varepsilon, 1) \cdots (q_n, \varepsilon, n)\bar{g}_n(\bar{q}_n, \varepsilon, n)(q_{n+1}, \varepsilon, n+1)$ of $\mathcal{B}$ over u , which concludes the proof of correctness. $\square$

5.4. Pebble weighted automata and weighted logic

The following theorem summarizes the main results of this section.

THEOREM 5.11. *Let $\mathbb{S}$ be a commutative semiring and $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$. The following assertions are equivalent over $\mathbb{S}$ and A :*

- (1) f is $\text{wFOTC}^b(\text{FO})$ -definable;
- (2) f is $\text{wFOTC}^{b,<}(\text{FO})$ -definable;
- (3) f is recognizable by a P2WA;
- (4) f is recognizable by a P1WA.

Inclusions $2 \Rightarrow 1$ and $4 \Rightarrow 3$ follow from the definitions of weighted logics and pebble weighted automata. Proposition 5.13 will reveal that $1 \Rightarrow 3$. Proposition 5.9 will prove that $3 \Rightarrow 4$: notice that this is the only implication using the commutativity of the semiring. The theorem follows from Proposition 5.12, which finally proves implication $4 \Rightarrow 2$.

PROPOSITION 5.12. *Every series recognizable by a P1WA is $\text{wFOTC}^{b,<}(\text{FO})$ -definable.*

PROOF. Starting from a P1WA $\mathcal{A}$ with p pebbles, we build a $\text{wFOTC}^{b,<}(\text{FO})$ -formula by induction on p . Without loss of generality, we may assume that lift transitions only take place at the end marker $\triangleleft$. The case $p = 0$ follows from Theorem 4.1 since a P1WA with 0 pebble can be translated easily into a weighted automaton by removing the first transitions reading $\triangleright$, and *modulo* is definable in $\text{wFOTC}^{b,<}(\text{FO})$ (see Example 3.13). Note that the tape contains $\triangleright u \triangleleft$ and $\widetilde{\text{pos}}(u) = \{0, \dots, |u| + 1\}$ but we still have $\text{pos}(u) = \{1, \dots, |u|\}$, $\text{first} = 1$ and $\text{last} = |u|$.

The induction step is shown like in the proof of the implication $1 \Rightarrow 2$ of Theorem 4.1. Notice that we no longer suppose that there exists only one initial state $q_0 = 1$. The main difference is however in the definition of the formulas $\Xi_{q,q'}^v(x)$, which now uses the induction hypothesis. When dropping the *first* pebble on some position z , the head is reset to the initial position labeled $\triangleright$, then the automaton behaves as a P1WA with $(p - 1)$ pebbles, starting in some state q , reading the word with position z “marked”, until it reaches the final position labeled $\triangleleft$ in some states q' , where it will have to lift the first pebble. By induction, there is a $\text{wFOTC}^{b,<}(\text{FO})$ -formula $\Theta_{q,q'}(z)$ computing the sum of the weights of all these runs of $\mathcal{A}$.

For $\ell \geq 1$, $v = v_1 \cdots v_\ell \in A^+$ and $q_0, q_\ell \in Q$, we define

$$\Xi_{q_0, q_\ell}^v(x) \stackrel{\text{def}}{=} \bigoplus_{q_1, \dots, q_{\ell-1} \in Q} \bigotimes_{1 \leq k \leq \ell} P_{v_k}(x - q_0 + k) \otimes \nu_{q_k, v_{k+1}, q_{k+1}}(x - q_0 + k)$$

with

$$\begin{aligned} \nu_{q,a,q'}(z) &\stackrel{\text{def}}{=} \text{weight}(q, a, \emptyset, \rightarrow, q') \oplus \bigoplus_{r, r', r'' \in Q} \\ &\quad (\text{weight}(q, a, \emptyset, \text{drop}, r) \otimes \Theta_{r,r'}(z) \otimes \text{weight}(r', \triangleleft, \emptyset, \text{lift}, r'') \otimes \text{weight}(r'', a, \emptyset, \rightarrow, q')). \end{aligned}$$

We have to guess non-deterministically the states $q_1, \dots, q_{\ell-1}$ that the automaton will visit between positions x and y while pebble p is not dropped, because each of them is useful to compute the runs with pebble p dropped. We obtain for $u \in A^+$ and positions $i, i + \ell - 1 \in \text{pos}(u)$

$$\llbracket \Xi_{q_0, q_\ell}^{u[i..i+\ell-1]} \rrbracket(u, i + q_0 - 1) = \sum_{q_1, \dots, q_{\ell-1} \in Q} \prod_{0 \leq k \leq \ell-1} \llbracket \nu_{q_{k-1}, u_{i+k}, q_k}(z) \rrbracket(u, i + k).$$

Then, we use almost the same weighted formulas as in Theorem 4.1. For small words $u \in A^+$ with $|u| \leq 2m$, we use

$$\Phi_{\text{small}} \stackrel{\text{def}}{=} (x = \text{first} \wedge y = \text{last}) \otimes \bigoplus_{1 \leq d \leq 2m, v \in A^d} \left[(\text{last} = d) \wedge \bigwedge_{1 \leq k \leq d} P_{v_k}(k) \right] \otimes \llbracket \mathcal{A} \rrbracket(v)$$

and we obtain $\llbracket \Phi_{\text{small}} \rrbracket(u) = \llbracket \mathcal{A} \rrbracket(u)$. Finally, we define

$$\Phi_1(x, y) = \bigoplus_{\substack{q_0 \in I, q, q' \in Q \\ v \in A^m}} (\text{first} = x \wedge 2m < \text{last} \wedge y = m + q') \otimes \text{weight}(q_0, \triangleright, \emptyset, \rightarrow, q) \otimes \Xi_{q, q'}^v(q)$$

$$\Phi_2(x, y) = \bigoplus_{q, q' \in Q, v \in A^m} (\text{first} < x \wedge x - q + 2m < \text{last} \wedge x \equiv_m q \wedge y - q' = x - q + m) \otimes \Xi_{q, q'}^v(x)$$

$$\Phi_3(x, y) = \bigoplus_{\substack{q \in Q, q' \in F \\ m < d \leq 2m, v \in A^d}} (\text{first} < x \wedge x - q + d = \text{last} \wedge x \equiv_m q \wedge y = \text{last}) \otimes \Xi_{q, q'}^v(x)$$

and we can show, as in the proof of Theorem 4.1, that $[\text{TC}_{x, y}^{2m, <}(\Phi_{\text{small}} \oplus \Phi_1 \oplus \Phi_2 \oplus \Phi_3)](\text{first}, \text{last})$ computes $\llbracket \mathcal{A} \rrbracket$, as required.

We remark that this construction yields a $\text{wFOTC}^{\text{b}, <}(\text{FO})$ -expression using $p + 2$ nested $\text{TC}^{N, <}$ -operators: $p + 1$ nested $\text{TC}^{N, <}$ -operators come from the induction in the proof of Theorem 4.1, and the last one comes from the definition of the modulo operation. $\square$

PROPOSITION 5.13. *Let $\mathbb{S}$ be a (non necessarily commutative) semiring and $f \in \mathbb{S}\langle\langle A^+ \rangle\rangle$. If f is $\text{wFOTC}^{\text{b}}(\text{FO})$ -definable, then f is recognizable by a P2WA.*

PROOF. By Lemma 3.3, we know that every series which is $\text{wFOTC}^{\text{b}}(\text{FO})$ -definable is in fact $\text{wFOTC}^{\text{b}}(\text{AP})$ -definable. Hence, for the unweighted part of $\text{wFOTC}^{\text{b}}(\text{FO})$, we just have to prove that atomic formulas $P_a(x)$, $\neg P_a(x)$, $x \leq y$, $\neg(x \leq y)$ are recognized by some P2WA with 0 pebble. For $P_a(x)$ (respectively, $\neg P_a(x)$), the automaton simply scans the input word, searching for letter $(a, 1)$ (respectively, $(b, 1)$ with $b \neq a$), as the free variable x is encoded in the alphabet $A \times \{0, 1\}$. For the atomic formula $\neg(x \leq y)$, the automaton scans the input word, looking for the encoding of y before that of x . One verifies the formula $x \leq y$ in the same way.

It remains to prove that the class P2WA is closed under the weighted constructs of $\text{wFOTC}^{\text{b}}(\text{FO})$. First, notice that the constant series $s \in \mathbb{S}$ is easily recognized with a P2WA with 0 pebble. Note that if a series is recognizable by a P2WA with p pebbles, it is also recognizable by a P2WA with $p + 1$ pebbles. Therefore, given two P2WAs, one can assume that they use the same number of pebbles. Let $\mathcal{A}_1, \mathcal{A}_2$ be two P2WAs with p pebbles over A . Closure under $\oplus$ is obtained using the disjoint union of the automata $\mathcal{A}_1$ and $\mathcal{A}_2$.

The P2WA with p pebbles that recognizes the Hadamard product which maps every word $u \in A^+$ to $\llbracket \mathcal{A}_1 \rrbracket(u) \times \llbracket \mathcal{A}_2 \rrbracket(u)$ consists of three phases: first, it simulates the automaton $\mathcal{A}_1$ until it reaches the last position in a final state of $\mathcal{A}_1$; then, it comes back to the beginning of the word with transitions of weight 1; finally, it simulates the automaton $\mathcal{A}_2$ and exit in a final state of $\mathcal{A}_2$.

For first-order quantifiers, we use an extra pebble. For a P2WA $\mathcal{A}$ with p pebbles over alphabet $A \times \{0, 1\}$ computing $\llbracket \Phi(x) \rrbracket$, we construct two P2WA with $(p + 1)$ pebbles over alphabet A computing respectively $\llbracket \bigoplus_x \Phi \rrbracket$ and $\llbracket \bigotimes_x \Phi \rrbracket$.

The automaton for $\bigoplus_x \Phi$ nondeterministically chooses with transitions of weight 1 a position in the word, drops the extra pebble $p + 1$, and simulates automaton $\mathcal{A}$, interpreting pebble $p + 1$ as the extra $\{0, 1\}$ component encoding variable x . After this simulation, it lifts the pebble and terminates the computation, going to the endmarker $\triangleleft$ with weights 1.

For $\bigotimes_x \Phi$, with transitions of weight 1, the new automaton drops successively the extra pebble $p+1$ on positions $1, \dots, |u|$ of the input word $u \in A^+$. Whenever it drops pebble $p+1$, it simulates $\mathcal{A}$ interpreting this pebble as the encoding of variable x until it reaches the end marker $\triangleleft$ where it lifts pebble $p+1$.

Finally, assume that we have a P2WA $\mathcal{A}$ with p pebbles over the alphabet $A \times \{0, 1\}^2$ which evaluates some wFOTC^b(FO) formula $\Phi(x, y)$. We construct a P2WA $\mathcal{A}'$ with $(p+2)$ pebbles over $A \times \{0, 1\}^2$ in order to evaluate $[\text{TC}_{x,y}^N \Phi](x', y')$ (x' and y' are now the two free variables). With transitions of weight 1, $\mathcal{A}'$ moves to the position i' to which maps variable x' . If y' also maps to i' , it simply simulates $\mathcal{A}$ interpreting $x' = y'$ as $x = y$, resulting in the evaluation of $\Phi(i', i')$. Otherwise, it drops its new pebble $p+2$. Whenever $\mathcal{A}'$ drops pebble $p+2$ on some position i , it enters some special state q_{drop} and with weights 1, it moves to some guessed position $j \neq i$ such that $|j - i| \leq N$ and drops pebble $p+1$. Whenever $\mathcal{A}'$ drops pebble $p+1$, it then simulates $\mathcal{A}$ interpreting positions i and j on which pebbles $p+2$ and $p+1$ are dropped as x and y , resulting in the evaluation of $\Phi(i, j)$. When the simulation of $\mathcal{A}$ is completed, $\mathcal{A}'$ lifts pebble $p+1$ with weight 1 in order to return to position j . Still with weights 1, $\mathcal{A}'$ moves to position i where pebble $p+2$ was dropped remembering $j - i$ and lifts pebble $p+2$. It then returns to position j . If $y' = j$ is the current position, the transitive closure is completed. With weights 1, we move to the end marker $\triangleleft$ and accept. Otherwise, $\mathcal{A}'$ drops pebble $p+2$ again in order to continue the evaluation of the transitive closure.

Note that, the simplicity requirement on runs implies that pebble $p+2$ cannot be dropped twice on the same position. Indeed, after dropping pebble $p+2$, $\mathcal{A}'$ always enters the same state q_{drop} . Moreover, pebble $p+2$ is never dropped on position y' . On a successful run ρ of $\mathcal{A}'$, either pebble $p+2$ is never dropped, and then we have $x' = i' = y'$ and we compute $\Phi(i', i')$; otherwise, pebble $p+2$ is successively dropped on some positions $x' = i_0, i_1, \dots, i_{\ell-1}$ such that $\ell \geq 1$ and $x' = i_0, i_1, \dots, i_{\ell-1}, i_\ell = y'$ are pairwise distinct and $|i_j - i_{j-1}| \leq N$ for $0 \leq j < \ell$. The sum of the weights of the successful runs of $\mathcal{A}'$ visiting such a pairwise distinct sequence $x' = i_0, \dots, i_\ell = y'$ is precisely $\llbracket \Phi \rrbracket(u, i_0, i_1) \times \dots \times \llbracket \Phi \rrbracket(u, i_{\ell-1}, i_\ell)$. We deduce that $\mathcal{A}'$ computes the N -bounded transitive closure $[\text{TC}_{x,y}^N \Phi](x', y')$. $\square$

6. ALGORITHMIC ISSUES

6.1. Evaluation

An important problem is the evaluation of an automaton wrt. a given word. More precisely, given a word $u = u_1 \dots u_n$ and a P1WA $\mathcal{A} = (Q, A, I, F, \Delta, \text{weight})$, we are interested in computing $\llbracket \mathcal{A} \rrbracket(u)$. In the sequel, we assume as before that lift transitions in $\mathcal{A}$ only occur at the end marker $\triangleleft$.

We fix the word $u = u_1 \dots u_n \in A^+$ and inductively compute (using dynamic programming) a matrix $M(\pi, i) \in \mathbb{S}^{Q \times Q}$ for $0 \leq i \leq n+1$, and $\pi \in \text{pos}(u)^{\leq p}$: coefficient $M(\pi, i)_{q,q'}$ is the sum of weights of runs that go from position 0 in state q to position i in state q' with pebbles $p, \dots, p - |\pi| + 1$ permanently dropped on positions described by π . Let $\mu(a, P, d) \in \mathbb{S}^{Q \times Q}$ be the matrix defined by $\mu(a, P, d)_{q,q'} = \text{weight}(q, a, P, d, q')$ if $(q, a, P, d, q') \in \Delta$, and $\mu(a, P, d)_{q,q'} = 0$ otherwise. We denote by $Id \in \{0, 1\}^{Q \times Q}$ the identity matrix and we initially set $M(\pi, 0)_{q,q'} = Id$. For $1 \leq i \leq n+1$, we set

$$M(\pi, i) = \begin{cases} M(\pi, i-1) \times \mu(u_{i-1}, \text{peb}(\pi, i-1), \rightarrow) \times \\ (Id + \mu(u_i, \text{peb}(\pi, i), \text{drop}) \times M(\pi i, n+1) \times \mu(\triangleleft, \emptyset, \text{lift})) & \text{if } i \leq n \wedge |\pi| < p \\ M(\pi, i-1) \times \mu(u_{i-1}, \text{peb}(\pi, i-1), \rightarrow) & \text{otherwise.} \end{cases}$$

Indeed, going from position 0 to position i can be decomposed as first going from position 0 to position $i-1$, then performing a right move and finally optionally (we use the identity matrix to simplify the matrix computation in this formula) dropping a pebble. Then, we

obtain

$$\llbracket \mathcal{A} \rrbracket(u) = \lambda \times M(\varepsilon, n+1) \times \gamma$$

where $\lambda \in \{0, 1\}^{1 \times Q}$ is the line vector coding the initial states ($\lambda_q = 1$ if and only if $q \in I$) and $\gamma \in \{0, 1\}^{Q \times 1}$ is the column vector coding the final states ($\gamma_q = 1$ if and only if $q \in F$). Using dynamic programming, this can be computed in time $\mathcal{O}(|Q|^3 \cdot |u|^{p+1})$. Here, $\mathcal{O}(|Q|^3)$ is due to matrix multiplication, and $\mathcal{O}(|u|^{p+1})$ is the number of matrices $M(\pi, i)$ that have to be determined. We obtain:

THEOREM 6.1. *Evaluation of a P1WA with p pebbles and set of states Q wrt. a word $u \in A^+$ is $\mathcal{O}(|Q|^3 \cdot |u|^{p+1})$.*

This evaluation procedure can be improved with some efforts as presented in [Gastin and Monmege 2012]. In particular, lifted to our framework, this permits to evaluate a P1WA with $\mathcal{O}(|Q|^3 \cdot |u|^{\max(p,1)})$ scalar operations (sum and product of elements of the semiring).

6.2. Satisfiability/emptiness

In the context of formal power series, one may call a closed expression $\Phi \in \text{wFOTC}^{b, <}(\text{FO})$ *satisfiable* if there is a word $w \in A^+$ such that $\llbracket \Phi \rrbracket(w) \neq 0$. Over commutative positive semirings (i.e., zero-sum free and without divisors of zero), satisfiability is decidable due to Theorem 5.11, which reduces the problem to non-emptiness of the support of a formal power series recognized by a P2WA. The latter problem, in turn, can be reduced to the decidable emptiness problem for classical pebble automata over the boolean semiring, as stated, e.g., in [Bojańczyk 2008].

However, this problem turns undecidable over general (not necessarily positive) semiring. In fact, we prove undecidability of emptiness for P1WA with 2 pebbles.

THEOREM 6.2. *Emptiness is undecidable over the semiring $\mathbb{Z}$ for P1WA with 2 pebbles, and formulas of wFO(AP).*

PROOF. A complete 2-counter machine $\mathcal{M}$ is a tuple $(Loc, \Delta, \text{init}, F)$ with Loc a finite set of locations, $\Delta = Loc \times Instr \times Loc$ the set of transitions where $Instr = \{Inc^p, Dec^p, ZTest^p \mid p \in \{1, 2\}\}$, $\text{init} \in Loc$ the initial location and $F \subseteq Loc$ the set of final locations. If $w = (\ell_0, D_1, \ell_1) \cdots (\ell_{n-1}, D_n, \ell_n) \in \Delta^*$ and $D \in Instr$, we set $|w|_D = |\{k \in [1, n] \mid D_k = D\}|$, and $w_j = (\ell_0, D_1, \ell_1) \cdots (\ell_{j-1}, D_j, \ell_j)$ for $1 \leq j \leq n$. We define the value of counter $p \in \{1, 2\}$ after step $j \in \{1, \dots, n\}$ as $c_j^p(w) = |w_j|_{Inc^p} - |w_j|_{Dec^p}$, and we also set $c_0^p = 0$.

An *accepting run* of $\mathcal{M}$ is a word $w = (\ell_0, D_1, \ell_1) \cdots (\ell_{n-1}, D_n, \ell_n) \in \Delta^+$ such that:

- (1) $\ell_0 = \text{init}$ and $\ell_n \in F$,
- (2) $c_j^p(w) \geq 0$ for all $j \in \{1, \dots, n\}$ and $p \in \{1, 2\}$,
- (3) if $D_j = ZTest^p$, then $c_{j-1}^p(w) = 0$ for all $j \in \{1, \dots, n\}$ and $p \in \{1, 2\}$.

The emptiness problem for 2-counter machines consists in deciding whether a given 2-counter machine has an accepting run. It is well known to be undecidable [Minsky 1967]. This problem reduces to emptiness of 1-way pebble word automata with 2 pebbles over $(\mathbb{Z}, +, \times, 0, 1)$. From a 2-counter machine $\mathcal{M}$, we build such an automaton $\mathcal{A}$ assigning a nonzero weight to accepting runs of $\mathcal{M}$, and weight 0 to all other words. Hence, $\mathcal{A}$ has a nonzero semantics if and only if $\mathcal{M}$ has an accepting run.

A word $w \in \Delta^+$ is not an accepting run of $\mathcal{M}$ if and only if it does not consists in consecutive transitions, or it violates either (1), (2) or (3). Let $\mathcal{D}^p = \{j \leq |w| \mid D_j = Dec^p\}$ and $\mathcal{Z}^p = \{j \leq |w| \mid D_j = ZTest^p\}$. Then w violates (2) if and only if for some $p \in \{1, 2\}$,

there exists $j \in \mathcal{D}^p$ such that $c_{j-1}^p(w) = 0$, i.e.,

$$\prod_{j \in \mathcal{D}^p} c_{j-1}^p(w) = 0. \quad (8)$$

We can compute this product with a P1WA with 1 pebble over $\mathbb{Z}$ as explained in Example 5.5. Next, if (2) holds, then w violates (3) if and only if for some $p \in \{1, 2\}$, there exists $j \in \mathcal{Z}^p$ such that $c_{j-1}^p(w) > 0$, that is, if and only if

$$\prod_{j \in \mathcal{Z}^p} \prod_{1 < j} (c_{j-1}^p(w) - k) = 0. \quad (9)$$

We have seen in Example 5.5 how to compute this value with a 2-pebble P1WA over $\mathbb{Z}$.

Henceforth, on every position $j \in \mathcal{D}^1 \cup \mathcal{D}^2 \cup \mathcal{Z}^1 \cup \mathcal{Z}^2$, automaton $\mathcal{A}$ drops a pebble in order to compute the products (8) or (9) as explained in Example 5.5. More precisely,

- on positions $j \in \mathcal{D}^1 \cup \mathcal{D}^2$, a drop transition leads to a copy of $\mathcal{A}_{\text{non-neg}}$, highlighted in Figure 4.
- on positions $j \in \mathcal{Z}^1 \cup \mathcal{Z}^2$, a drop transition leads to a copy of the automaton $\mathcal{A}_{\text{zero-test}}$ highlighted in Figure 5.
- a loop is used to do nothing on every position $j \notin \mathcal{D}^1 \cup \mathcal{D}^2 \cup \mathcal{Z}^1 \cup \mathcal{Z}^2$.
- finally, transitions with weight 0 are used either on the loop- or the drop-transitions to check if (1) is violated or if two consecutive letters of Δ are not consecutive transitions in $\mathcal{M}$.

Using Example 3.6, it is easy to construct a formula of wFO(AP) equivalent to automaton $\mathcal{A}$, showing that emptiness is also undecidable for wFO(AP). $\square$

The problem is still open for automata with a single pebble.

7. CONCLUSION

In this paper, we introduced weighted automata with pebbles. We showed that these devices capture weighted first order logic extended with a transitive closure operator. Contrary to previous work, we do not restrict the first order logic but allow for arbitrary nesting of universal quantification. Moreover, we established several new characterizations of the classical recognizable formal power series in terms of logics with transitive closure.

These results are not only of theoretical interest. They also lay the basis for quantitative extensions of database query languages, such as XPath, and provide tracks to evaluate quantitative aspects of XML documents. The framework of weighted automata is natural for answering questions such as “How many nodes are selected by a request?”, or “How difficult is it to answer a query?”. The navigational mechanism of pebble automata is also well-suited in this context. So, our work is a first step before tackling similar questions on trees.

A second, and maybe related, line of future research concerns quantitative model checking, aiming at a general framework of quantitative specification languages that cover existing ones like [Laroussinie et al. 2010a; Laroussinie et al. 2010b; Fischer et al. 2010]. Note that this would also involve branching structures rather than words, as they naturally represent unfoldings of Kripke structures.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous referees for their valuable comments that permitted to simplify some proofs, and globally improve the quality of this paper.

REFERENCES

- J. Berstel and Ch. Reutenauer. 2010. *Noncommutative Rational Series With Applications*. Cambridge University Press, Cambridge.
- M. Blum and C. Hewitt. 1967. Automata On a 2-Dimensional Tape. In *Proceedings of Switching and Automata Theory*.
- M. Bojańczyk. 2008. Tree-Walking Automata, In LATA'08. *Language and Automata Theory and Applications: Second International Conference (LATA 2008)* 5196 (2008), 1–17.
- M. Bojańczyk, M. Samuelides, T. Schwentick, and L. Segoufin. 2006. Expressive power of pebble automata. In *Automata, Languages and Programming (LNCS)*, Vol. 4051. Springer, 157–168.
- B. Bollig and P. Gastin. 2009. Weighted versus Probabilistic Logics. In *Proceedings of the 13th International Conference on Developments in Language Theory (DLT'09) (LNCS)*, Vol. 5583. Springer, 18–38.
- B. Bollig, P. Gastin, B. Monmege, and M. Zeitoun. 2010. Pebble Weighted Automata and Transitive Closure Logics. In *Proceedings of ICALP'10 (LNCS)*, Vol. 6199. Springer, 587–598. DOI:<http://dx.doi.org/10.1007/978-3-642-14162-1>
- J. R. Büchi. 1960. Weak Second-Order Arithmetic and Finite Automata. *Zeitschrift für mathematische Logik und Grundlagen der Mathematik* 6 (1960).
- H. Comon, M. Dauchet, R. Gilleron, C. Löding, F. Jacquemard, D. Lugiez, S. Tison, and M. Tommasi. 2008. *Tree Automata Techniques and Applications*. Available at <http://tata.gforge.inria.fr/>.
- M. Droste and P. Gastin. 2009. Weighted automata and weighted logics. See Droste et al. [2009], Chapter 5, 175–211.
- M. Droste, W. Kuich, and H. Vogler. 2009. *Handbook of Weighted Automata*. Springer.
- M. Droste and H. Vogler. 2006. Weighted tree automata and weighted logics. *Theoretical Computer Science* 366, 3 (2006), 228–247.
- M. Droste and H. Vogler. 2011. Weighted Logics for Unranked Tree Automata. *Theory Comput. Syst.* 48, 1 (2011), 23–47.
- C. C. Elgot. 1961. Decision problems of finite automata design and related arithmetics. *Trans. Amer. Math. Soc.* 98 (1961), 21–52.
- J. Engelfriet and H. J. Hoogeboom. 1999. Tree-Walking Pebble Automata. In *Jewels Are Forever, Contributions to Theoretical Computer Science in Honor of Arto Salomaa*. Springer, 72–83.
- J. Engelfriet and H. J. Hoogeboom. 2007. Automata with nested pebbles capture first-order logic with transitive closure. *Logical Methods in Computer Science* 3 (2007), 1–27.
- K. Etessami. 1997. Counting Quantifiers, Successor Relations, and Logarithmic Space. *J. Comput. Syst. Sci.* 54, 3 (1997), 400–411.
- I. Fichtner. 2011. Weighted Picture Automata and Weighted Logics. *Theory of Computing Systems* 48, 1 (2011), 48–78.
- D. Fischer, E. Grädel, and L. Kaiser. 2010. Model Checking Games for the Quantitative Mu-Calculus. *Theory Comput. Syst.* 47, 3 (2010), 696–719.
- P. Gastin and B. Monmege. 2012. Adding Pebbles to Weighted Automata. In *Proceedings of CIAA'12 (LNCS)*, Vol. 7381. Springer-Verlag, 28–51. Long version at <http://hal.archives-ouvertes.fr/hal-00753513>.
- H. Hansson and B. Jonsson. 1994. A Logic for Reasoning about Time and Reliability. *Formal Aspects of Computing* 6, 5 (1994), 512–535.
- L. Hella, L. Libkin, J. Nurmonen, and L. Wong. 2001. Logics with aggregate operators. *J. ACM* 48, 4 (2001), 880–907.
- J. E. Hopcroft and J. D. Ullman. 1979. *An Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley.
- N. Immerman and E. Lander. 1990. Describing Graphs: a First-Order Approach to Graph Canonization. In *Complexity Theory Retrospective*. Springer-Verlag.
- F. Laroussinie, A. Meyer, and E. Pettonnet. 2010a. Counting CTL. In *FOSSACS (Lecture Notes in Computer Science)*, C.-H. Luke Ong (Ed.), Vol. 6014. Springer, 206–220.
- F. Laroussinie, A. Meyer, and E. Pettonnet. 2010b. Counting LTL. In *TIME*, Nicolas Markey and Jef Wijsen (Eds.). IEEE Computer Society, 51–58.
- L. Libkin. 2000. Logics with counting and local properties. *ACM Trans. Comput. Log.* 1, 1 (2000), 33–59.
- C. Mathissen. 2010. Weighted Logics for Nested Words and Algebraic Formal Power Series. *Logical Methods in Computer Science* 6, 1 (2010).

- I. Meinecke. 2006. Weighted Logics for Traces. In *CSR (Lecture Notes in Computer Science)*, Vol. 3967. Springer, 235–246.
- M. L. Minsky. 1967. *Computation: finite and infinite machines*. Prentice-Hall, Inc., Upper Saddle River, NJ, USA.
- F. Neven and T. Schwentick. 2000. On the Power of Tree-Walking Automata. In *Proceedings of ICALP'00 (LNCS)*, Vol. 1853. Springer, 547–560.
- F. Neven, T. Schwentick, and V. Vianu. 2004. Finite State Machines for Strings over Infinite Alphabets. *ACM Transactions on Computational Logic* 5 (July 2004), 403–435. Issue 3. DOI:<http://dx.doi.org/10.1145/1013560.1013562>
- M. O. Rabin. 1969. Decidability of second-order theories and automata on infinite trees. *Trans. Amer. Math. Soc.* 141 (1969), 1–35.
- M. O. Rabin and D. Scott. 1959. Finite Automata and Their Decision Problems. *IBM Journal of Research and Development* 3, 2 (1959), 114–125.
- M. Samuelides and L. Segoufin. 2007. Complexity of Pebble Tree-Walking Automata. In *Proceedings of FCT'07*. Springer-Verlag, 458–469.
- M. P. Schützenberger. 1961. On the definition of a family of automata. *Information and Control* 4 (1961), 245–270.
- John C. Shepherdson. 1959. The Reduction of Two-Way Automata to One-Way Automata. *IBM Journal of Research and Development* 3, 2 (1959), 198–200.
- B. ten Cate and L. Segoufin. 2010. Transitive Closure Logic, Nested Tree Walking Automata, and XPath. *J. ACM* 57, 3 (2010), 18:1–18:41.
- J. W. Thatcher and J. B. Wright. 1968. Generalized Finite Automata Theory with an Application to a Decision Problem of Second-Order Logic. *Mathematical Systems Theory* 2, 1 (1968), 57–81.
- W. Thomas. 1982. Classifying Regular Events in Symbolic Logic. *J. Comput. System Sci.* 25, 3 (1982), 360–376.
- B. A. Trakhtenbrot. 1961. Finite automata and logic of monadic predicates. *Doklady Akademii Nauk SSSR* 149 (1961), 326–329. In Russian.